SOCKET PROGRAMMING BASICS

Byeong-hee Roh
Dept. of Software and Computer Engineering
Ajou University

변화와 융합의 중심
MMcN
멀티미디어 융합 연구실
Mobile Multimedia convergecne Network Lab.
http://mmcn.ajou.ac.kr

Contents

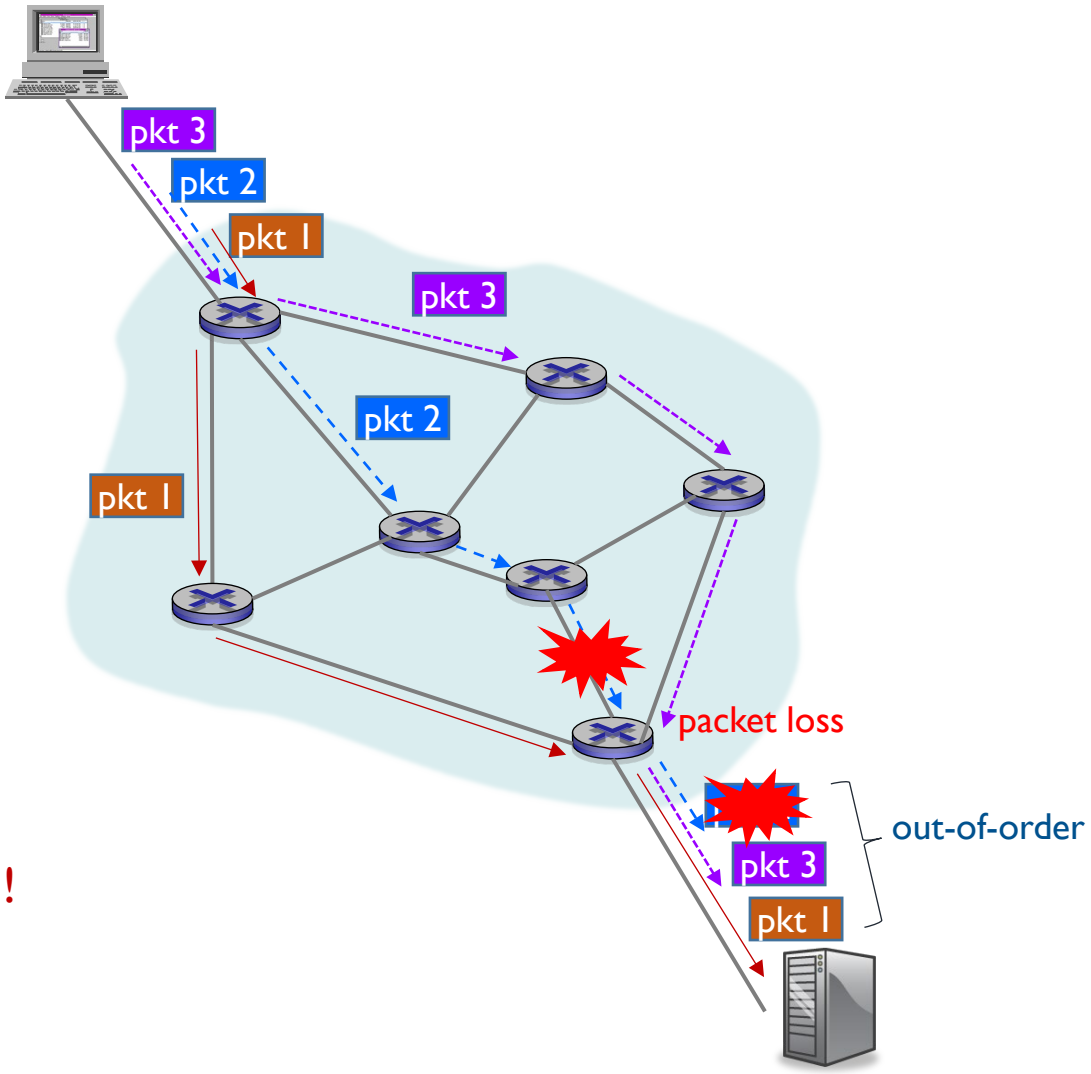
NETWORK APPLICATIONS

Need of Transport Services

Need of Transport Service

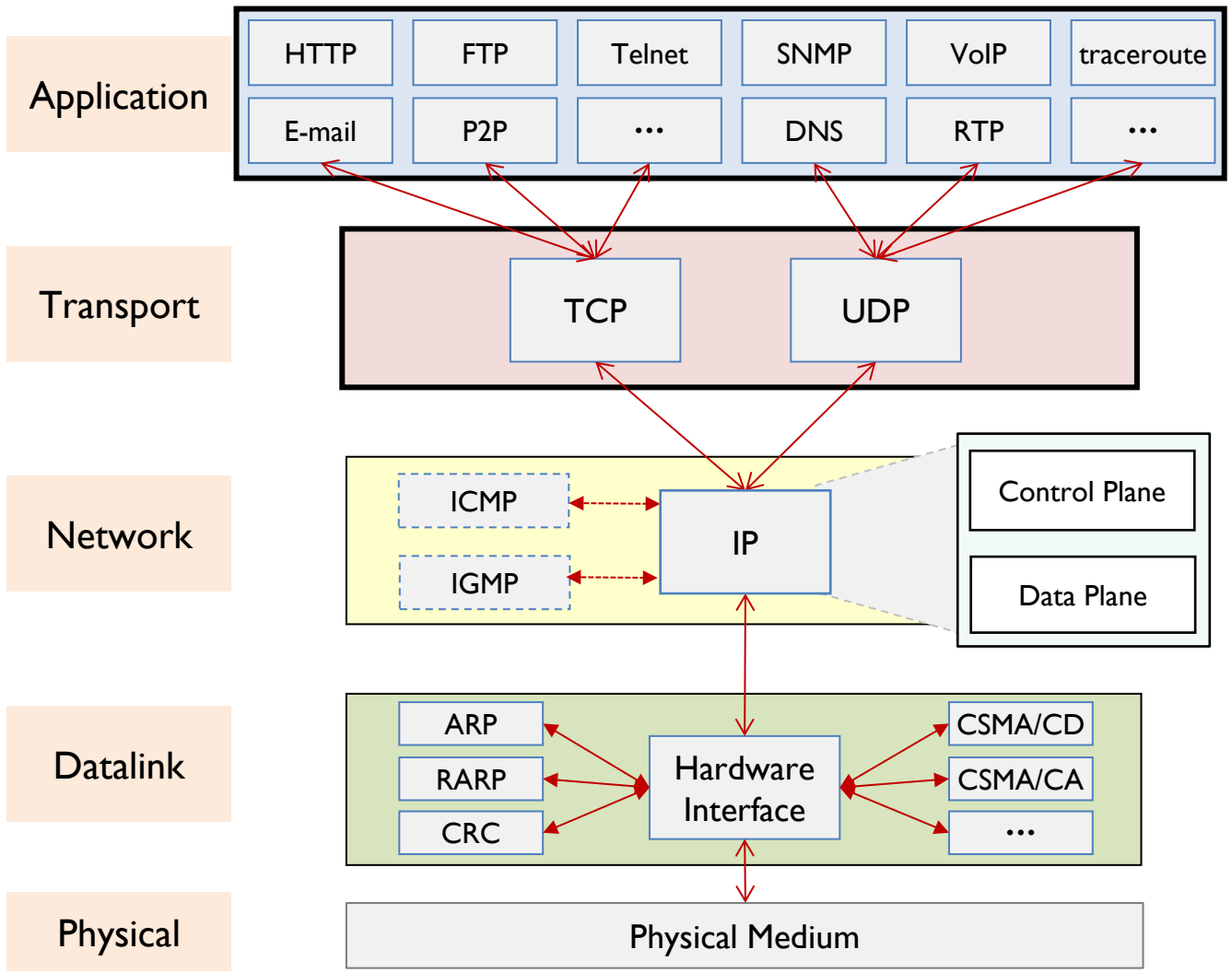
- Network Layer of the Internet
  - The Internet is based on datagram packet-switching technology
    - simple,
    - but connectionless
    - and unreliable
  - Each packet is treated independently
    - packets can take any practical route
    - routing decision for each packet is needed
    - packets may arrive out of order
    - packets may go missing

Unreliable !!



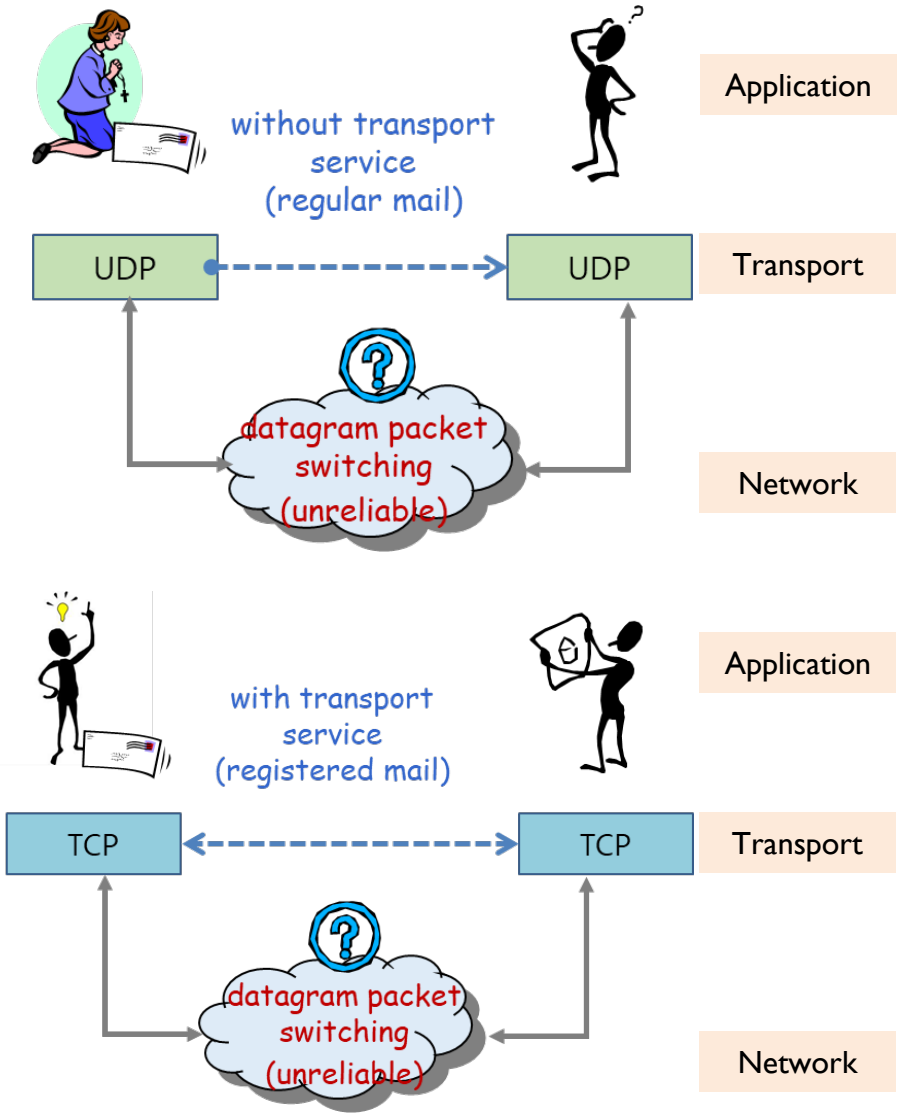
Need of Transport Service

- Application layer has
  - the only interface with transport layer
- Transport layer protocols
  - TCP
    - Transmission Control Protocol
  - UDP
    - User Datagram Protocol



Need of Transport Service

- Transport Layer Protocols
  - UDP (User Datagram Protocol)
    - connectionless
    - unreliable
    - out-of-order delivery
    - no-frills extension of "best-effort" IP
    - fast because no ACKs are used,
    - less traffic is sent across the network
  - TCP (Transmission Control Protocol)
    - connection-oriented
    - reliable
    - in-order delivery
    - congestion control
    - flow control
    - connection setup delay



Applications' Requirements

- data integrity (loss)
  - some apps require 100% reliable data transfer
    - ex) file transfer, web transactions
  - other apps can tolerate some loss
    - ex) realtime audio/video
- timing (delay)
  - some apps require low delay to be "effective"
    - ex) Internet telephony, interactive games
  - others can tolerate some delay
    - ex) file transfer, …
- throughput (bandwidth)
  - some apps require minimum amount of throughput to be "effective"
    - ex) multimedia, IPTV, …
  - other apps ("elastic apps") make use of whatever throughput they get
- security
  - encryption, data integrity, …

Transport Layer Services

❖ Requirements of transport services for some common apps

application	loss sensitivity	throughput	time sensitivity
file transfer	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5kbps-1Mbps video:10kbps-5Mbps	yes (≤ 100's msec)
stored audio/video	loss-tolerant	audio: 5kbps-1Mbps video:10kbps-5Mbps	yes (≤ few secs)
interactive games	loss-tolerant	few kbps up	yes (≤ 100's msec)
text messaging	no loss	elastic	no (but, sometimes yes)

Transport Layer Services

❖ Application and transport protocols for Internet apps

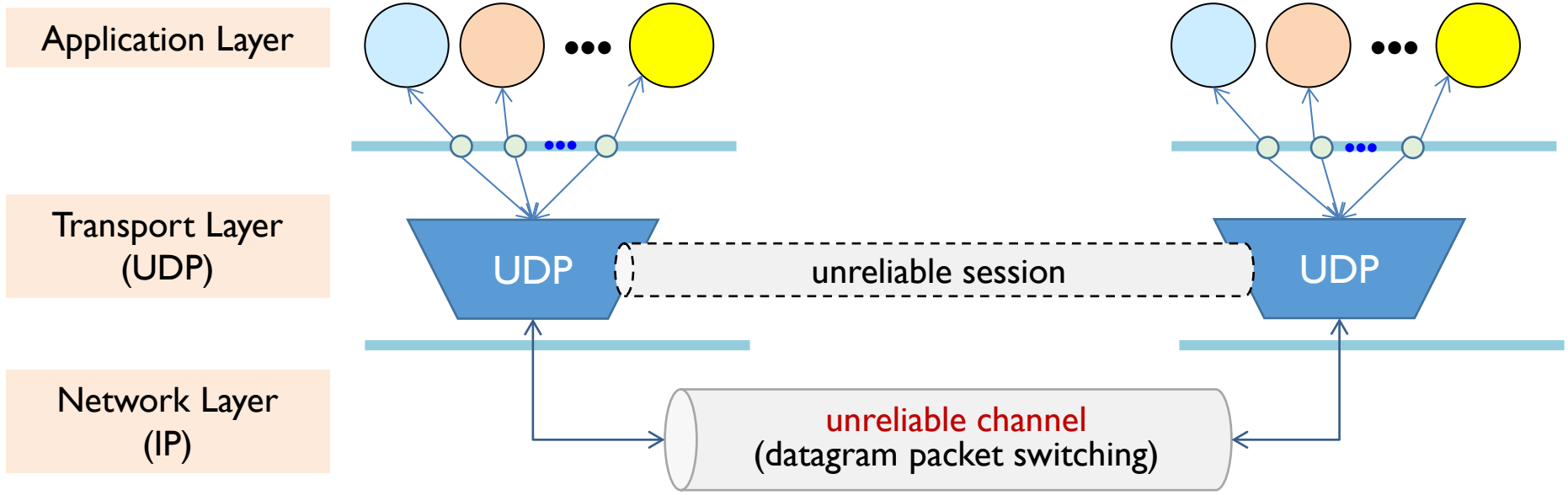
application	application layer protocol	underlying transport protocol
e-mail	SMTP [RFC 2821]	TCP
remote terminal access	telnet [RFC 854]	TCP
Web	HTTP [RFC 2616]	TCP
file transfer	FTP [RFC 959] HTTP [RFC 2616]	TCP
streaming multimedia	HTTP (e.g., YouTube), RTP [RFC 1889]	Typically TCP Typically UDP
Internet telephony	SIP, RTP, Proprietary (e.g., Skype)	Typically UDP (sometimes TCP)

UDP Socket Programming

UDP Overview

UDP – User Datagram Protocol

- UDP (User Datagram Protocol)
  - Primary service - multiplexing and demultiplexing
  - Connectionless (CL)
    - no-frills or bare-bones over IP at network layer → unreliable
    - lower overhead and fast

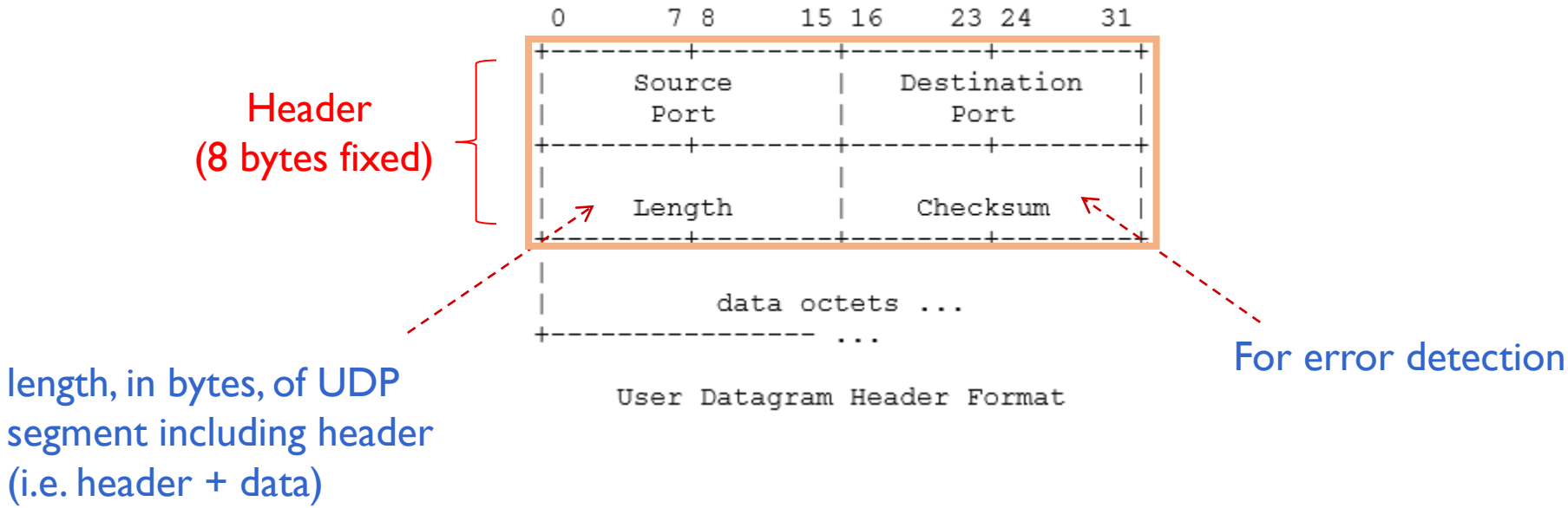


UDP - Features

- Advantages
  - simpler
  - flexible
  - efficient
  - fast
  - broadcast capability
- Disadvantages
  - unreliable
  - out-of-order delivery
  - message size limitation

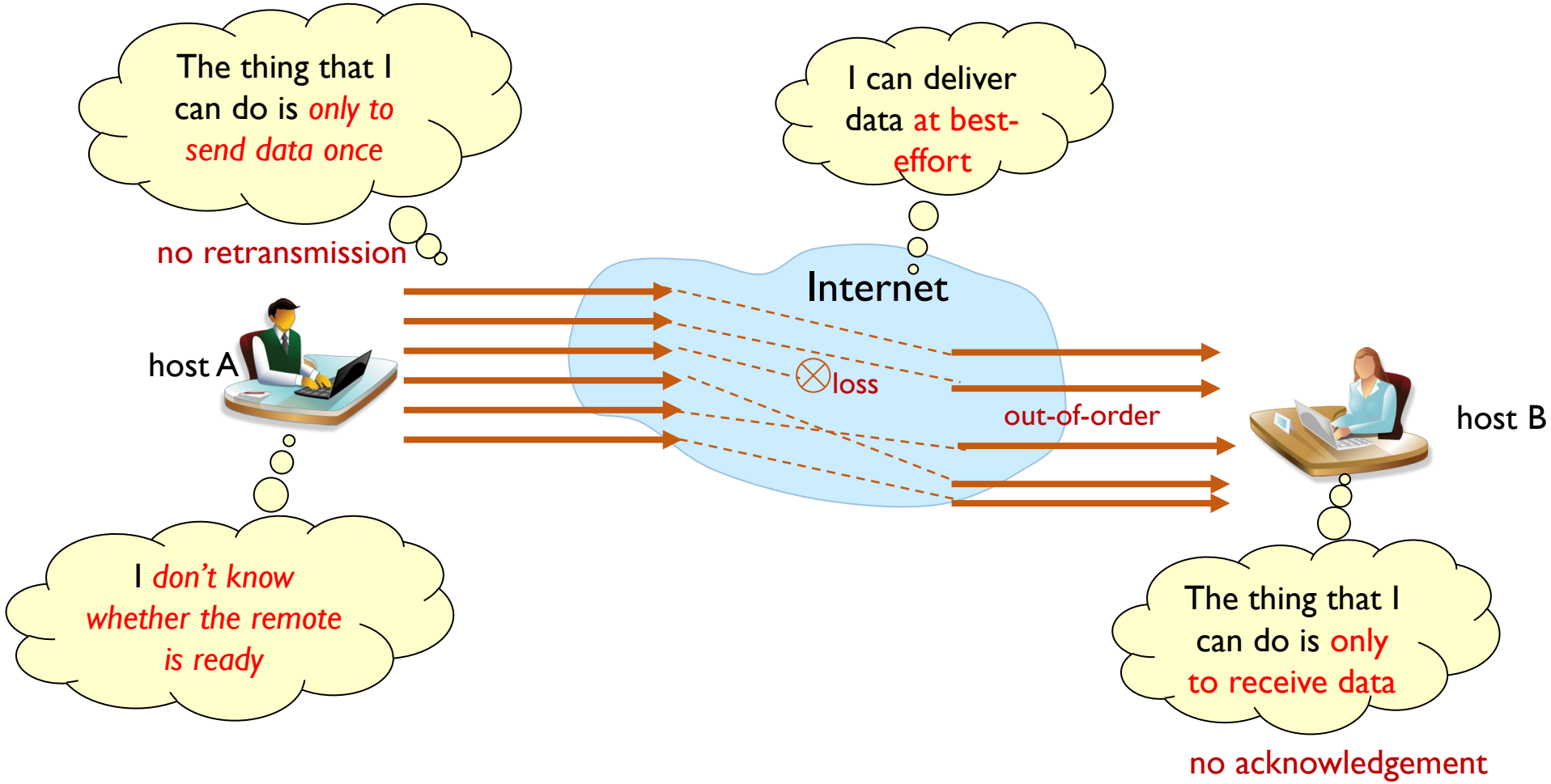
UDP – Segment Structure

❖ UDP Segment header



UDP – Connectionless (CL) Communications

❖ Features of CL communication



UDP – Connectionless (CL) Communications

❖ TCP vs. UDP

	TCP	UDP
1	• Connection-oriented (CO) Protocol	• Connectionless (CL) Protocol
2	• Communication after connection setup	• No connection setup
3	• Stream-oriented protocol - sequence number	• Message-oriented protocol - datagram
4	• Reliable data transfer - with timeout & retransmission	• Unreliable data transfer - no retransmission
5	• Unicast (1:1) only	• Unicast (1:1) • Broadcast (1:N) • Multicast (N:N)

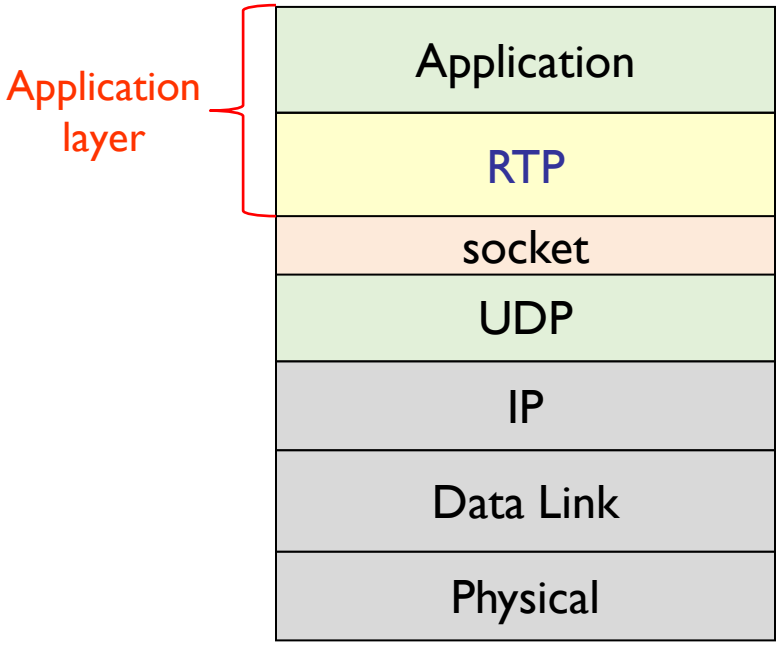
UDP Application Programming

- CL applications
  - no connection setup is needed
    - simple, flexible, efficient, fast, and broadcast capability
  - unreliable
    - out-of-order delay
    - missing
  - message size limitation (should be less than or equal to MSS)

- CL-based application programs should concern
  - lost packets
  - out-of-order delivery
  - message size limitation

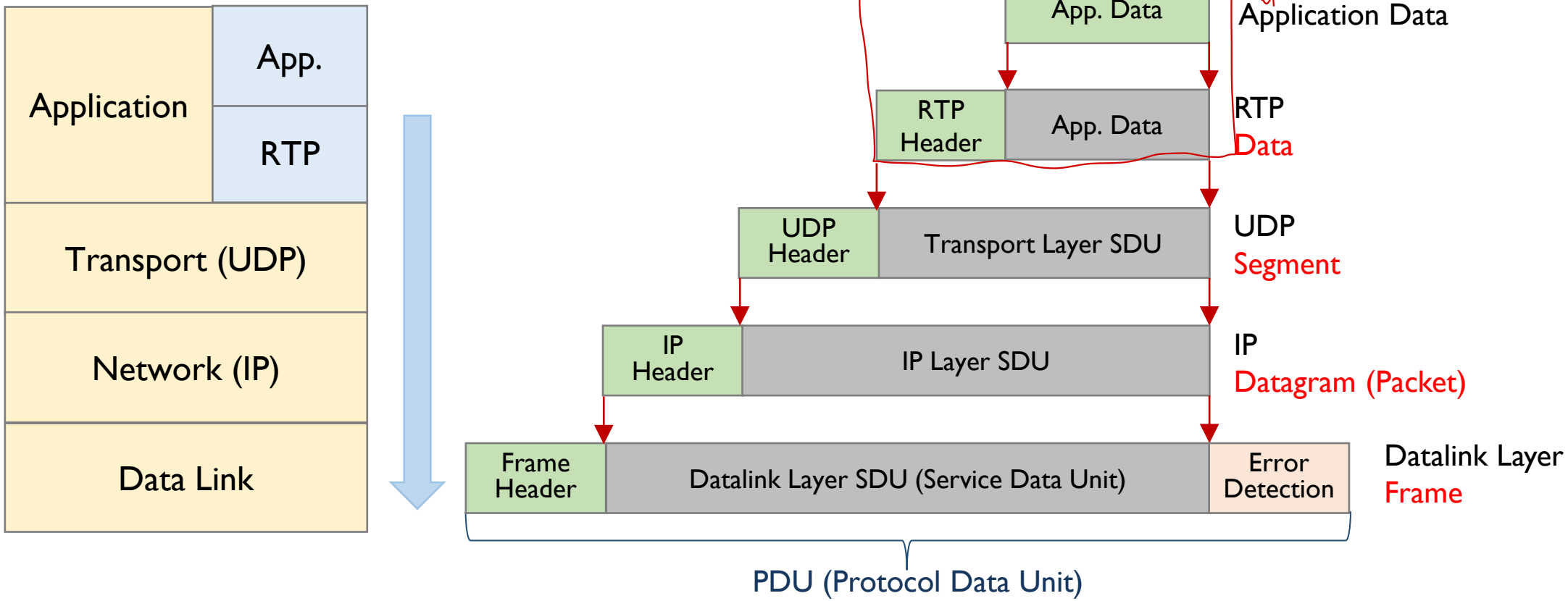
UDP-Supported Application Protocols: RTP

- Example 1: RTP for CL applications (1/3)
  - RTP (Realtime Transport Protocol)
    - A Transport Protocol for Real-Time Applications, RFC 3550, July 2003
    - Application layer protocol over UDP
  - RTP provides
    - End-to-end transport functions for real-time applications
    - Packet generation order information - sequence number
    - Synchronization between real-time medias - time stamp
    - Type of media (media-independent) - payload type
    - Multicast support - CSRC, SSRC
    - QoS feedback - RTCP
    - Packet encryption - padding
    - Multiplexing service - mixer, translator
    - Heterogeneous environments support - mixer, translator



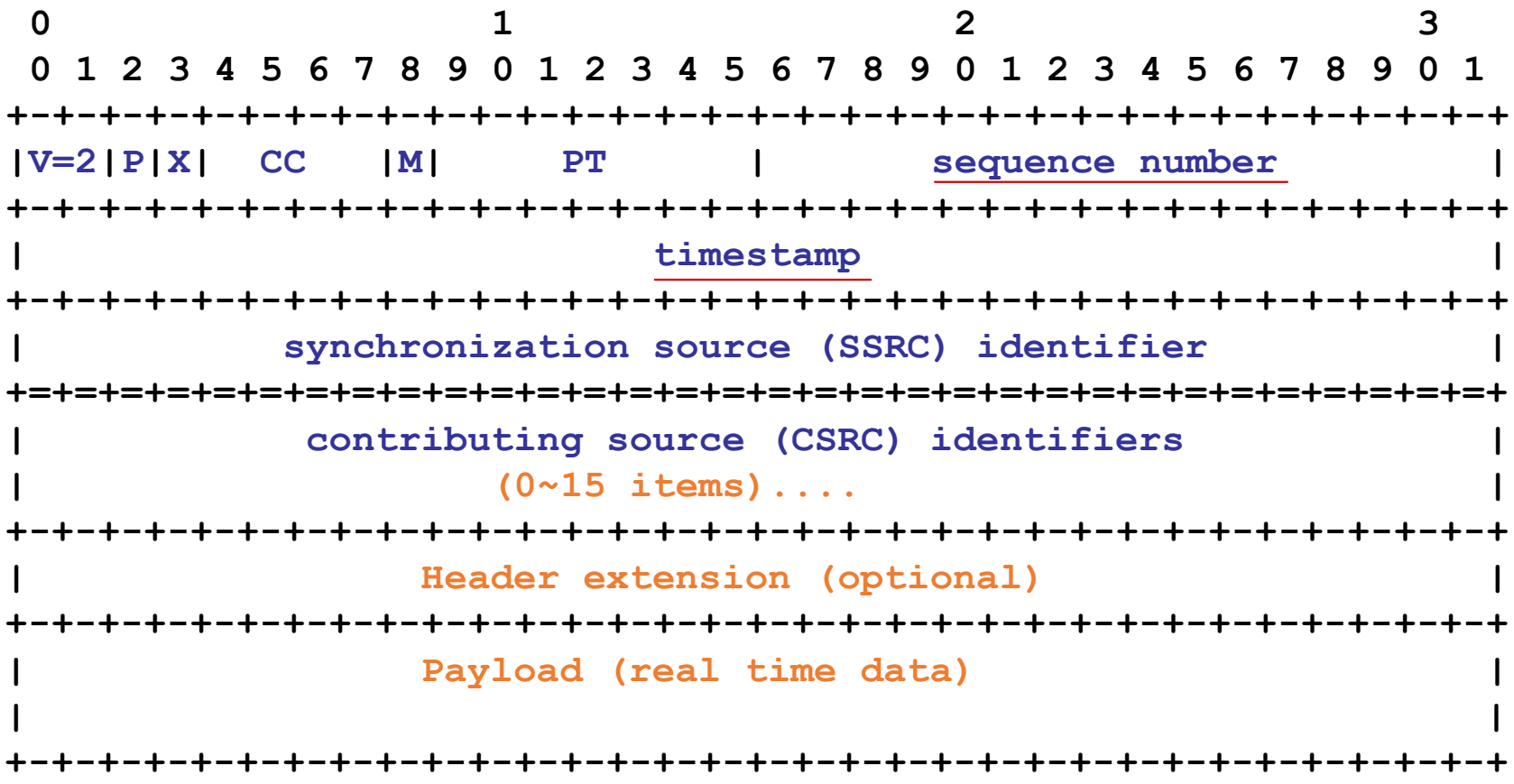
UDP-Supported Application Protocols: RTP

- Example 1: RTP for CL applications (2/3)
  - RTP Encapsulation
    - RTP is part of the application layer



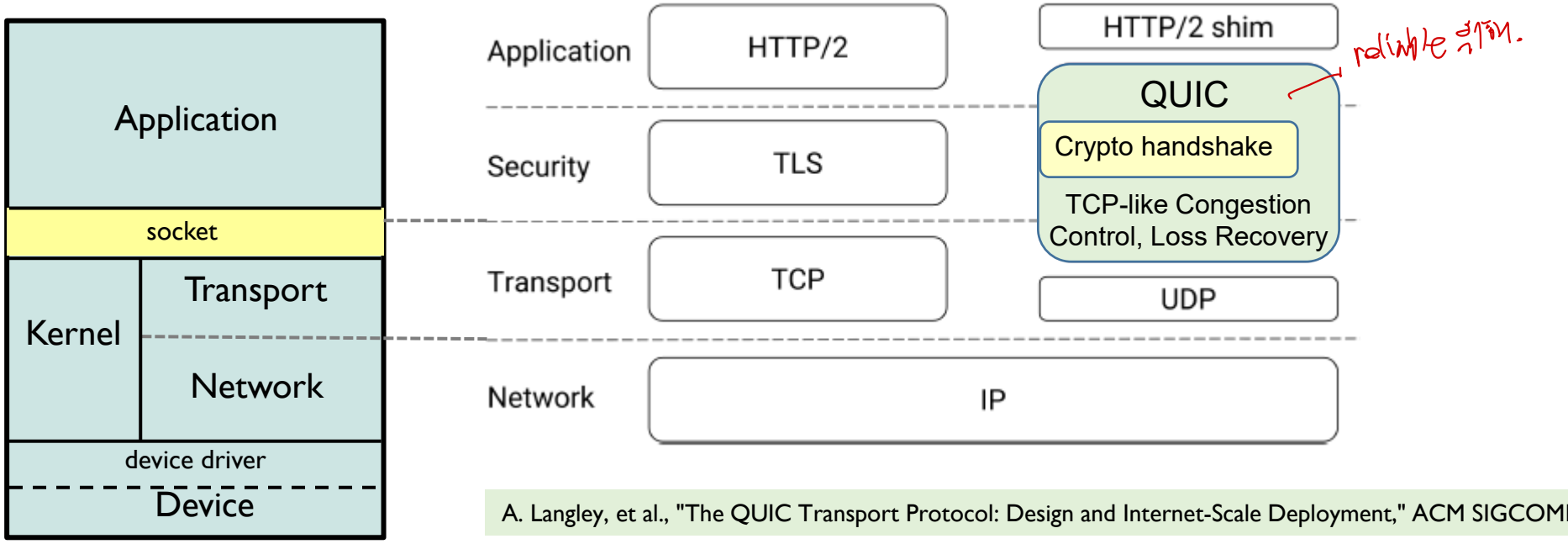
UDP-Supported Application Protocols: RTP

- Example 1: RTP for CL applications (3/3)
  - RTP Header



UDP-Supported Application Protocols: QUIC

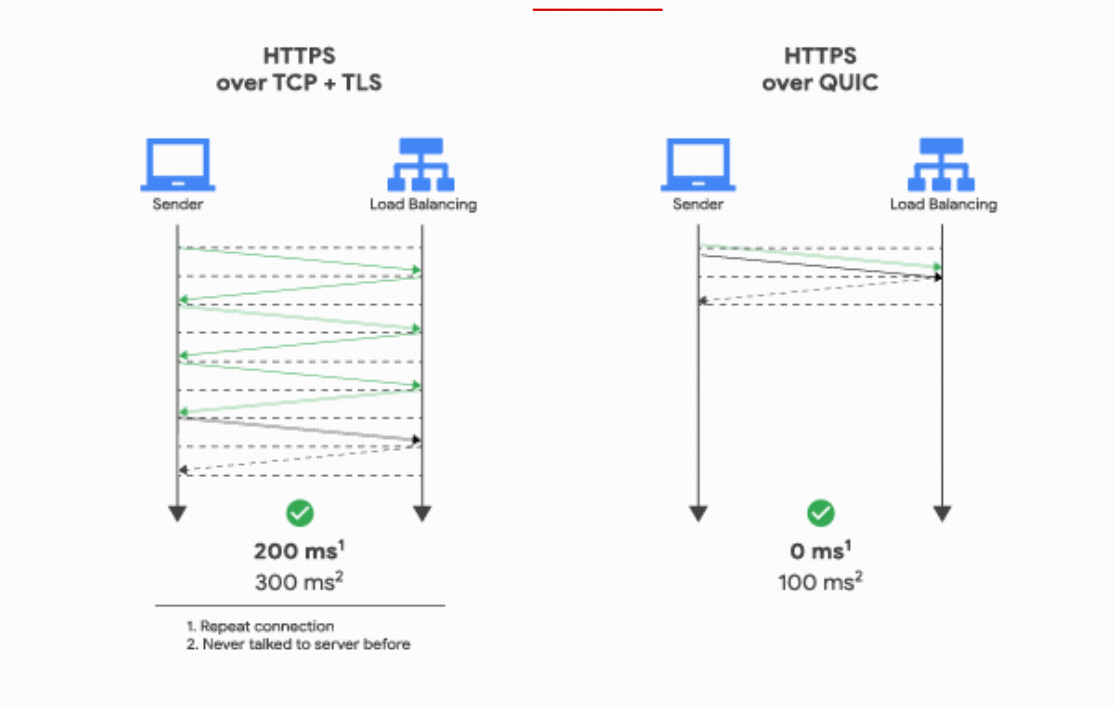
- Example 2: QUIC – Quick UDP Internet Connections (1/2)
  - a new transport protocol designed by Google in 2012
    - to improve performance for HTTPS traffic and
    - to make streaming better and faster
  - UDP-based, stream-multiplexing (TCP-like), encrypted transport protocol



A. Langley, et al., "The QUIC Transport Protocol: Design and Internet-Scale Deployment," ACM SIGCOMM '2017

UDP-Supported Application Protocols: QUIC

- Example 2: QUIC – Quick UDP Internet Connections (2/2)
  - Connection setup latency
    - HTTPS over TCP+TLS: 3 RTTs before HTTP message exchange
      - TCP connection – 1 RTT
      - TLS 1.2 connection – 2 RTT
    - HTTPS over QUIC: 0 RTT



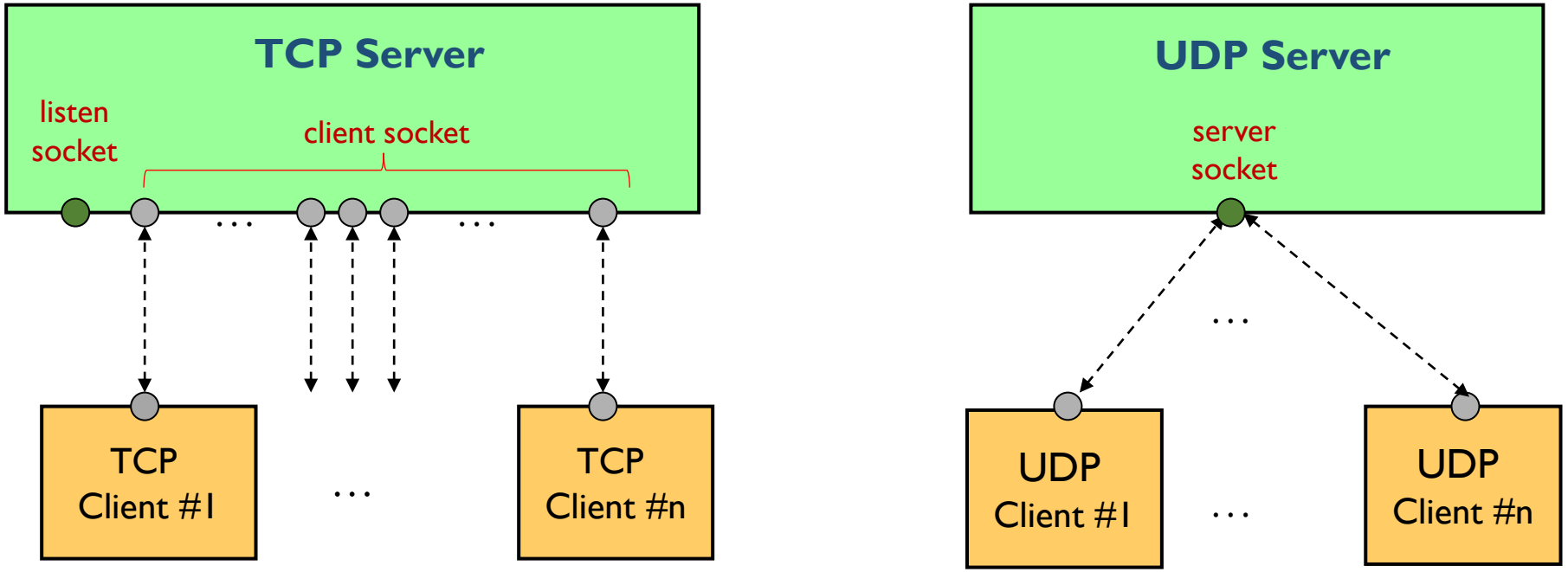
Source: Introducing QUIC support for HTTPS load balancing, https://cloudplatform.googleblog.com/2018/06/Introducing-QUIC-support-for-HTTPS-load-balancing.html

UDP Socket Programming

Socket Functions for UDP Application

Connectionless Communications

- Sockets for clients and servers on TCP and UDP
  - TCP server
    - one listen socket for connection setup
    - additional sockets for individual clients
  - UDP server
    - one socket for individual clients (no listen socket)



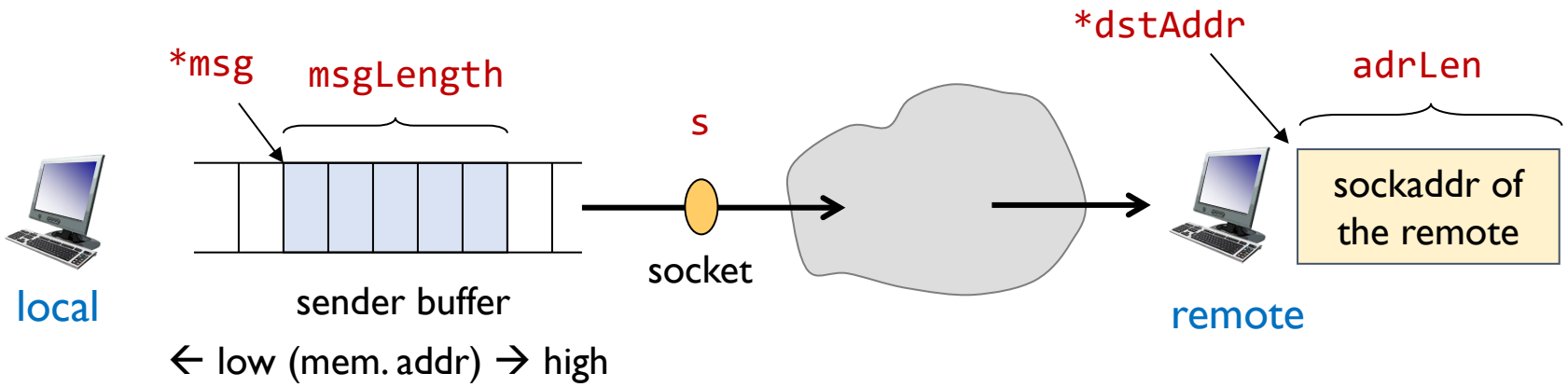
Socket Functions for CL Communications

- Two basic functions
  - sendto( )
    - at sender side
  - recvfrom( )
    - at receiver side

sendto( ) (1/3)

❖ sendto( ) function – send a message to a CL socket

```
int sendto ( int                     s,
             const void              *msg,
             int                     msgLength,
             unsigned                flags,
             const struct sockaddr   *destAddr,
             int                     addrLen );
```



sendto( ) (2/3)

- sendto( ) function (cont'd)
  - arguments
    - `s` : the socket number
    - `msg` : a pointer to the buffer holding the datagram
    - `msgLength` : the length of the datagram (in bytes)
    - `flags` : options for sending the datagram
    - `destAddr` : a pointer to a generic socket address of the remote
    - `addrLen` : the length of the address `destAddr`
  - return values
    - `-1`: when an error occurs
      - for the cause of the error, consult the value of errno
    - `the number of bytes sent`, when successful

sendto( ) (3/3)

❖ sendto( ) function (cont'd)

- flags

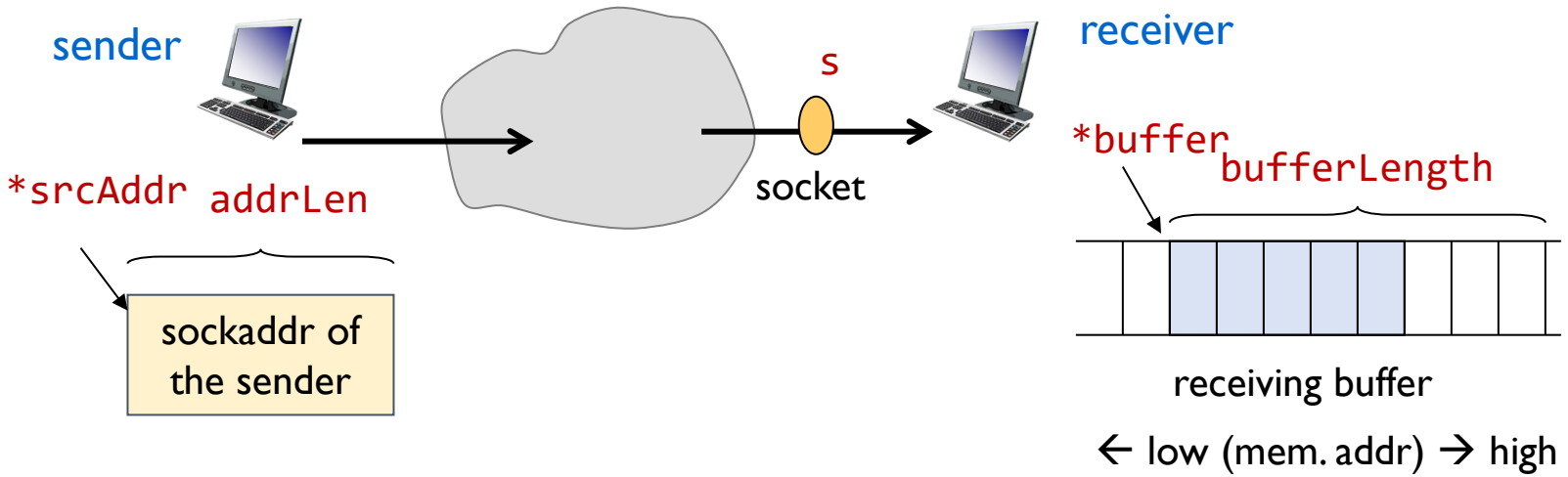
flag	meaning
0	Normal-no special options
MSG_OOB	Processes out-of-band data
MSG_DONTROUTE	Bypasses routing; uses direct interface
MSG_DONTWAIT	Does not block; waiting to write
MSG_NOSIGNAL	Does not raise SIGPIPE when the other end has disconnected

- though each flag has its own hexadecimal values,
- always use the macro names shown above.
- the hexadecimal value for each flag may be changed according to kernels.

recvfrom( ) (1/3)

❖ recvfrom( ) function – receive a message from a CL socket

```
int recvfrom (   int                 s,
                 void                *buffer,
                 int                 bufferLength,
                 unsigned            flags,
                 struct sockaddr     *srcAddr,
                 int                 *addrLen );
```



recvfrom( ) (2/3)

- recvfrom( ) function (cont'd)
  - arguments
    - `s` : the socket number
    - `buffer` : the buffer pointer holding the received datagram
    - `bufferLength` : maximum buffer size (in bytes)
      - *note that it is not the length of received datagram*
    - `flags` : options
    - `srcAddr` : a pointer to a generic socket address of the sender
    - `fromlen` : a pointer to the length of the address `srcAddr`
      - *note that the actual length of address is not given.*
      - *the actual length of address will be returned to the variable*
  - return values
    - `-1`: when an error occurs
      - for the cause of the error, consult the value of errno
    - `the number of bytes received`, when successful

recvfrom( ) (3/3)

- recvfrom( ) function (cont'd)
  - flags

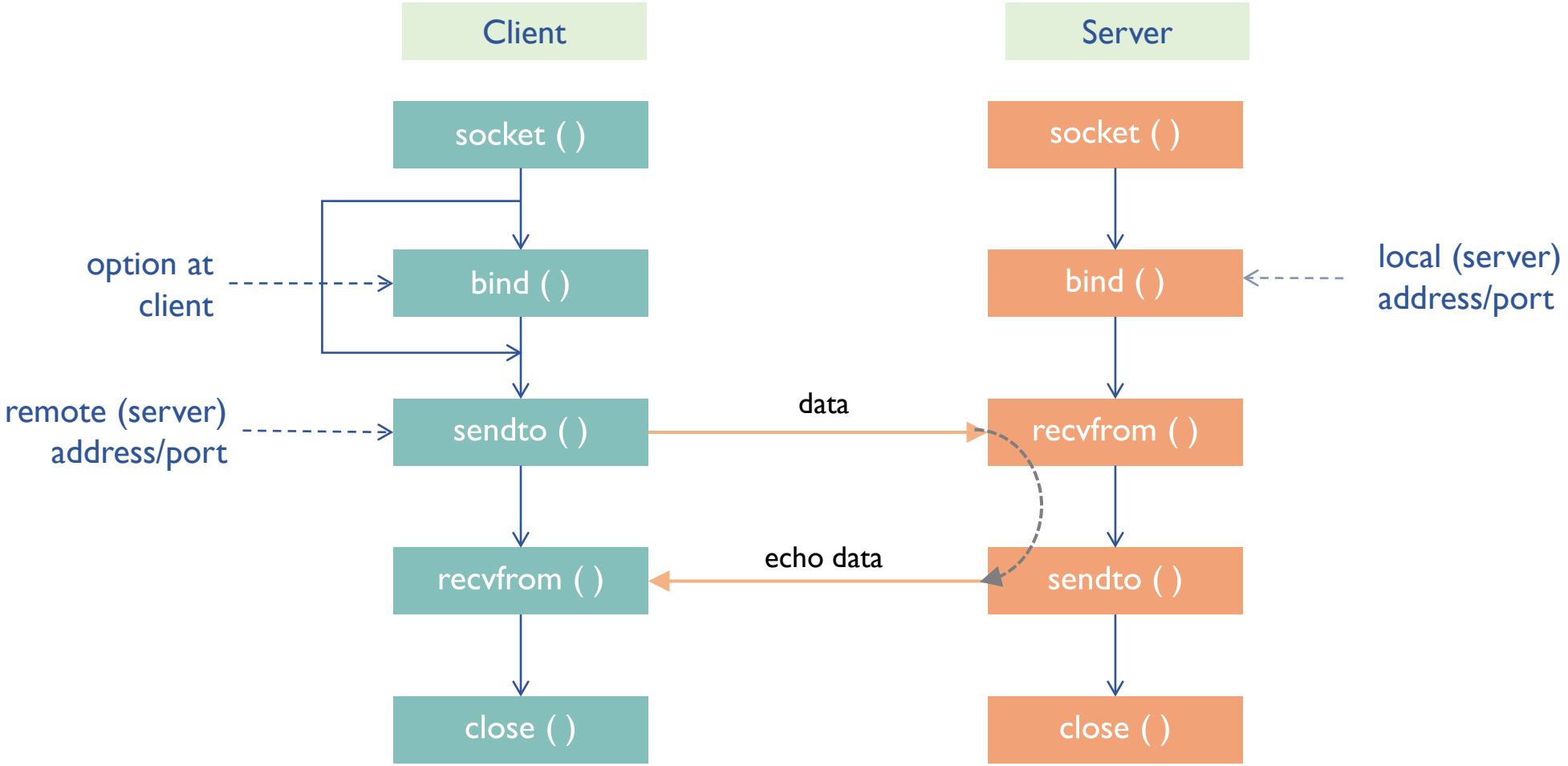
Flag	meaning
0	Normal-no special options
MSG_OOB	processes out-of-band data
MSG_PEEK	Reads a datagram without actually removing it from the kernel's receive queue
MSG_WAITALL	Requests that the operation block until the full request has been satisfied (with some exceptions)
MSG_ERRQUEUE	Receives a packet from the error queue
MSG_NOSIGNAL	Turns off the raising of SIGPIPE for stream sockets when the other end has become disconnected

UDP Socket Programming

Example 1: Echo Client-Server

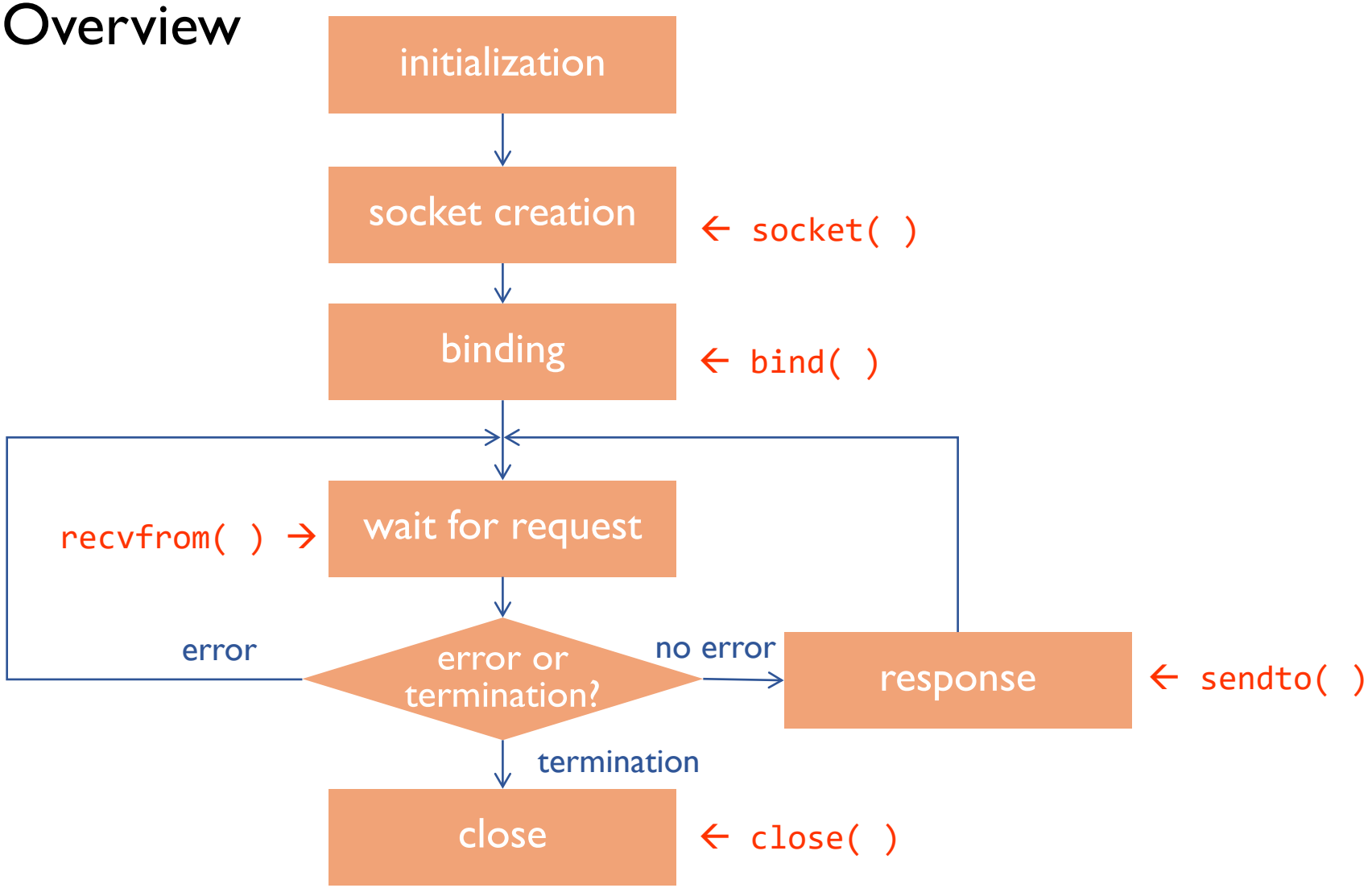
UDP Echo Client-Server

❖ Operation



UDP Echo Server

❖ Program Overview



UDPEchoServer.c – Example Code (1/2)

```
int                 sock;                   /* Socket */
struct sockaddr_in  echoServAddr;           /* Local address */
struct sockaddr_in  echoClntAddr;           /* Client address */
unsigned int        cliAddrLen;             /* incoming message Length*/
char                echoBuffer[ECHOMAX];    /* Buffer for echo string */
unsigned short      echoServPort;           /* Server port */
int                 recvMsgSize;            /* Size of received message */

// Create socket for sending/receiving datagrams
sock = socket (AF_INET, SOCK_DGRAM, 0);

// Construct local address structure
Memset (&echoServAddr, 0, sizeof(echoServAddr));
echoServAddr.sin_family      = AF_INET;
echoServAddr.sin_addr.s_addr = htonl (INADDR_ANY);    // Binding interface (network order)
echoServAddr.sin_port        = htons (echoServPort);  // Local port (network order)

/* Bind to the local address */
bind (sock, (struct sockaddr *) &echoServAddr, sizeof(echoServAddr));
```

UDPEchoServer.c – Example Code (2/2)

```
while (1 ) /* Run forever */
{
    cliAddrLen = sizeof(echoClntAddr);

    /* Block until receive message from a client */
    recvMsgSize = recvfrom ( sock, echoBuffer,  sizeof echoBuffer,  0,
                             (struct sockaddr *) &echoClntAddr,  &cliAddrLen ) ;

    if( recvMsgSize == SOCKET_ERROR ) continue;

    printf("Handling client %s\n", inet_ntoa (echoClntAddr.sin_addr));

    /* Send received datagram back to the client */
    sendto (sock,
            echoBuffer, recvMsgSize,  0,
            (struct sockaddr *) &echoClntAddr,
            sizeof(echoClntAddr)) ;
}
```

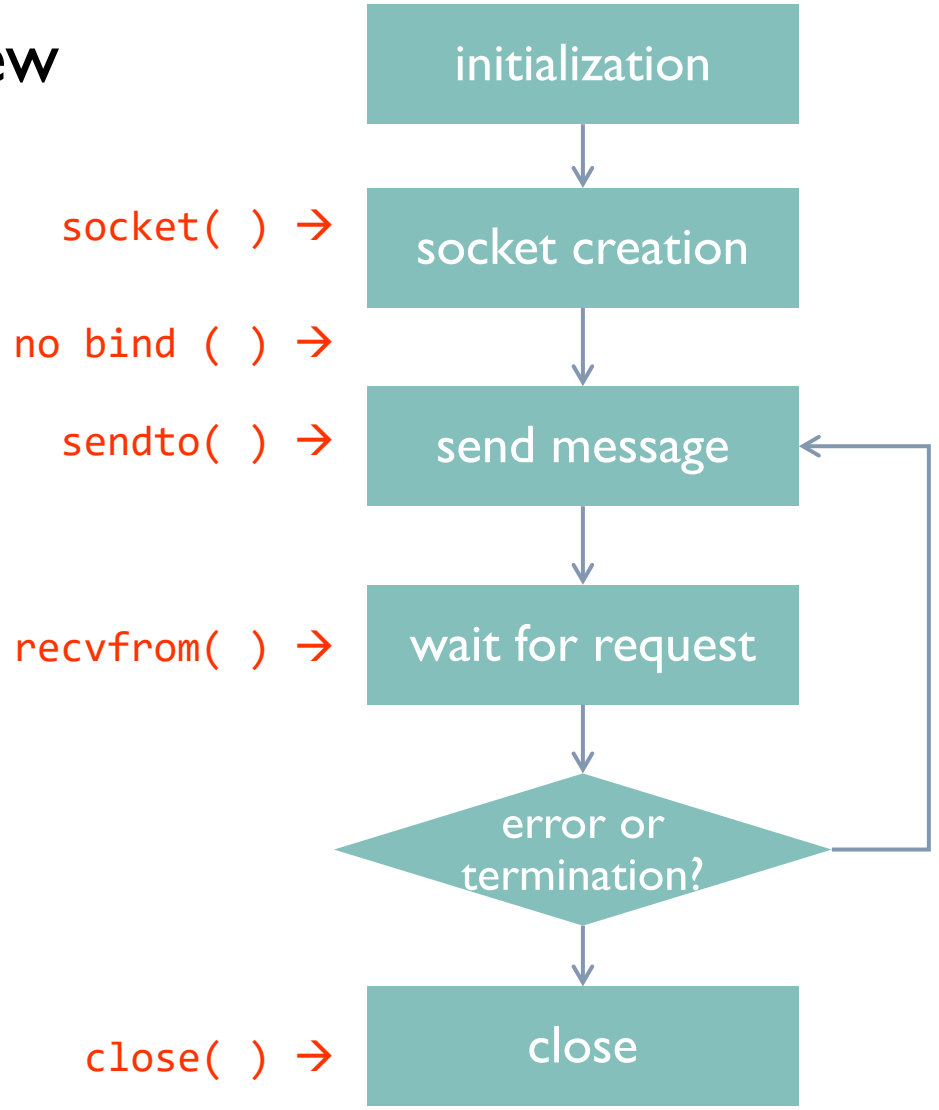
note: it should be pointer

client's address

note: it is not a pointer

UDP Echo Client

- Program Overview



UDPEchoClient.c – example code (1/2)

```
int                 sock;                   /* Socket descriptor */
struct sockaddr_in  echoServAddr;           /* Echo server address */
struct sockaddr_in  fromAddr;               /* Source address of echo */
unsigned short      echoServPort;           /* Echo server port */
unsigned int        fromSize;               /* In-out of address size*/
char                *servIP;                /* IP address of server */
char                *echoString;            /* String to send to server */
char                echoBuffer[ECHOMAX+1];  /* Receiving Buffer */
int                 echoStringLen;          /* Length of string to echo */
int                 respStringLen;          /* Length of received response */

servIP      = argv[1];              /* server IP address (dotted quad) */
echoString  = argv[2];              /* Second arg: string to echo */
echoServPort= atoi(argv[3]);        /* Use given port, if any */

/* Create a datagram/UDP socket */
sock = socket (AF_INET,  SOCK_DGRAM,  IPPROTO_UDP) ;
```

UDPEchoClient.c – example code (2/2)

```
memset(&echoServAddr, 0, sizeof(echoServAddr));
echoServAddr.sin_family = AF_INET;                    /* addr family */
echoServAddr.sin_addr.s_addr = inet_addr (servIP);    /* Server IP addr (network order)*/
echoServAddr.sin_port   = htons (echoServPort);       /* Server port (network order)*/

// no bind ( )

while ( 1 ) {
   -- routine for input the message to be sent (echoString)

   sendto ( sock, echoString, echoStringLen, 0,
            (struct sockaddr *) &echoServAddr, sizeof(echoServAddr) ) ;

   fromSize = sizeof(fromAddr);
   respStringLen = recvfrom (sock, echoBuffer, ECHOMAX, 0,
                             (struct sockaddr *) &fromAddr,  &fromSize)) ;
   if ( respStringLen == SOCKET_ERROR ) continue;

   -- routine for display the received message
}
close(sock);
```

it should be same

note: it is not a pointer

remote address

pointer

Discussion

- Note that
  - bind( ) was used in UDP server program,
  - while no call to bind( ) was made in UDP client pgm.

- Why can we leave out bind( ) in Client programs ?

TCP Socket Programming

TCP Overview

TCP – Connection-Oriented (CO)

❖ Definition of Connection-Oriented

sender 입장에서 receiver 입장에서 제대로 받았는지

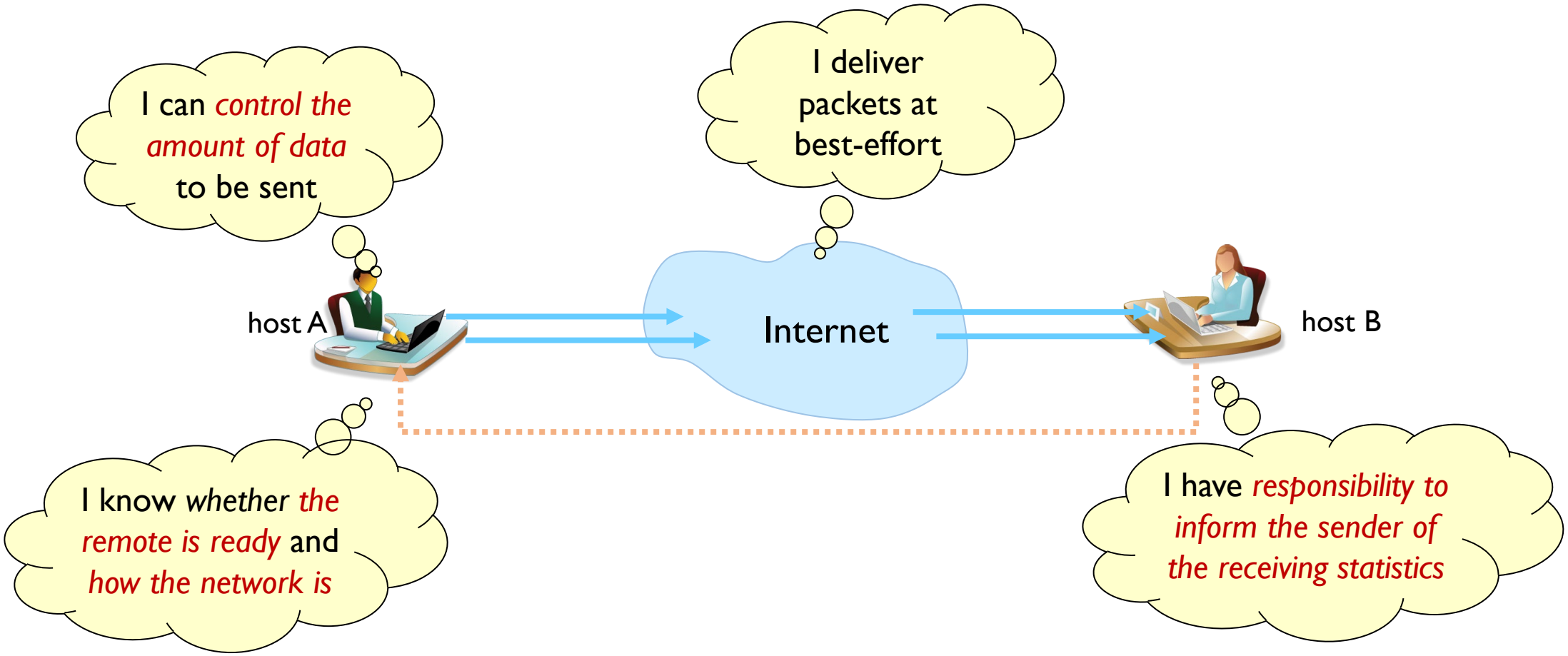
- Telephone Network
  - physical circuits' connection between end-to-end phones
  - no acknowledgement for received message at receiver
- TCP
  - no physical connection between end-to-end hosts
  - acknowledgement for received message at receiver

Telephone network
(by signaling and circuit switching)

TCP
(by positive acknowledgement by receiver)

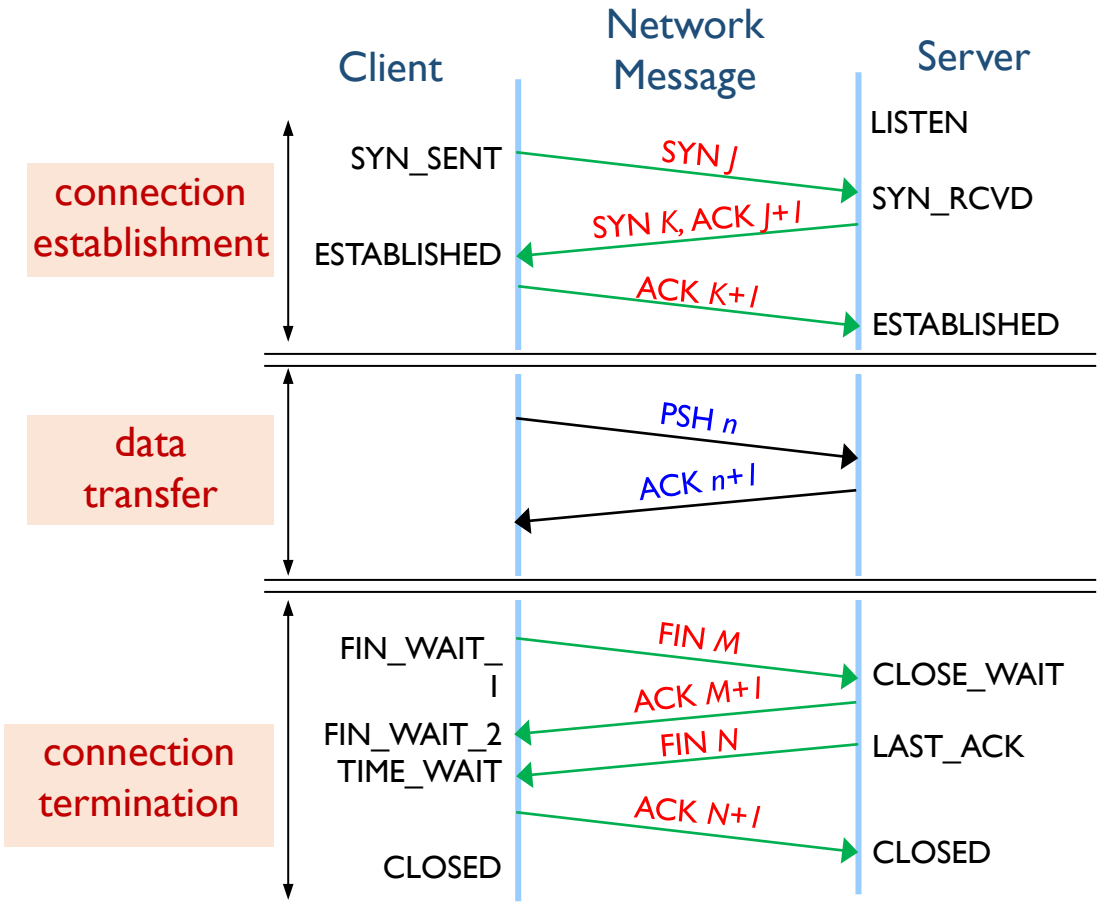
TCP – Connection-Oriented (CO)

❖ Features of CO communication



TCP – Connection-Oriented (CO)

- TCP is
  - connection-oriented
  - three phases
    - connection establishment
    - data transfer
    - connection termination
- TCP Connection
  - Relationship for positive ACK mechanism between the client and the server
  - not the physical connection between end hosts as in circuit switching network



TCP – Application Programming

- CO-based application programs can be written without the following problems:
  - lost packets
  - timeouts and retransmissions
  - duplicated packets
  - out-of-order delivery
  - flow control
  - no limitation on message (data) size

- Basic Requirement
  - connection establishment with a remote socket

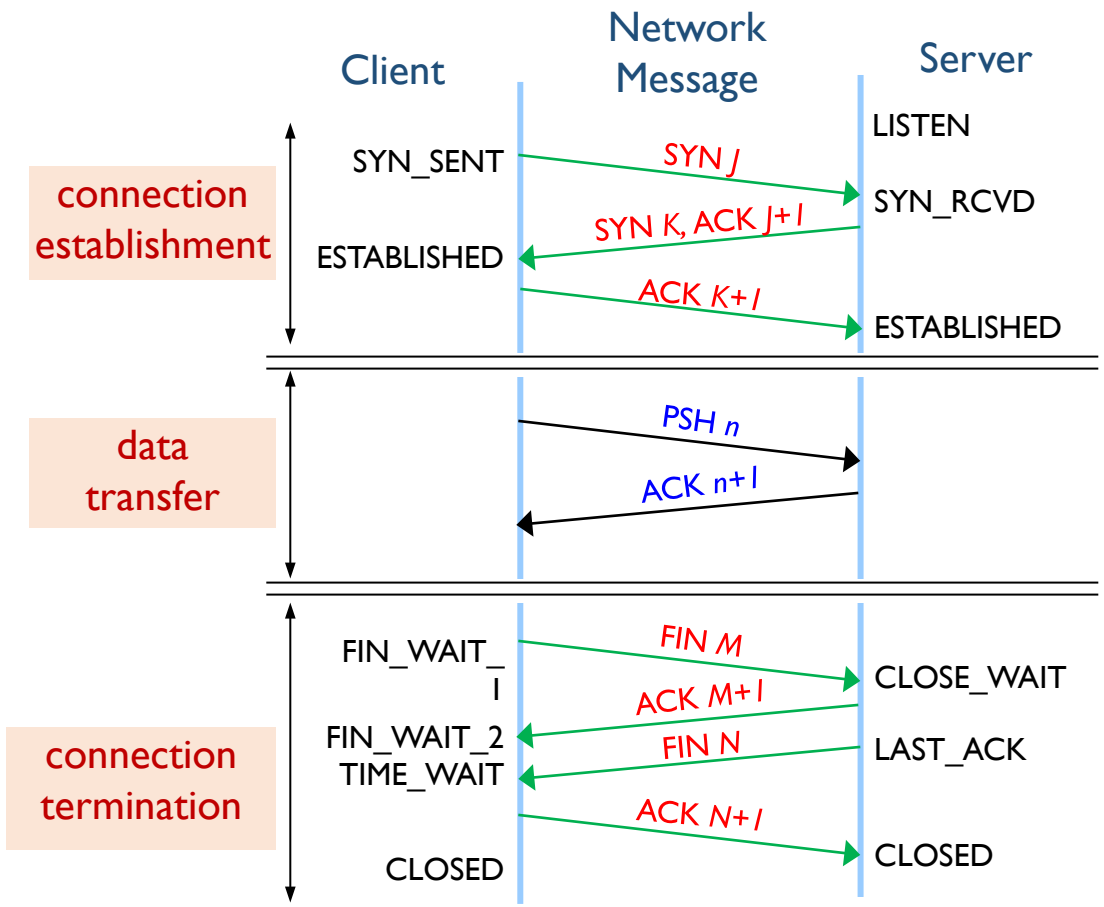
TCP – Operation

- TCP is connection-oriented
  - three phases

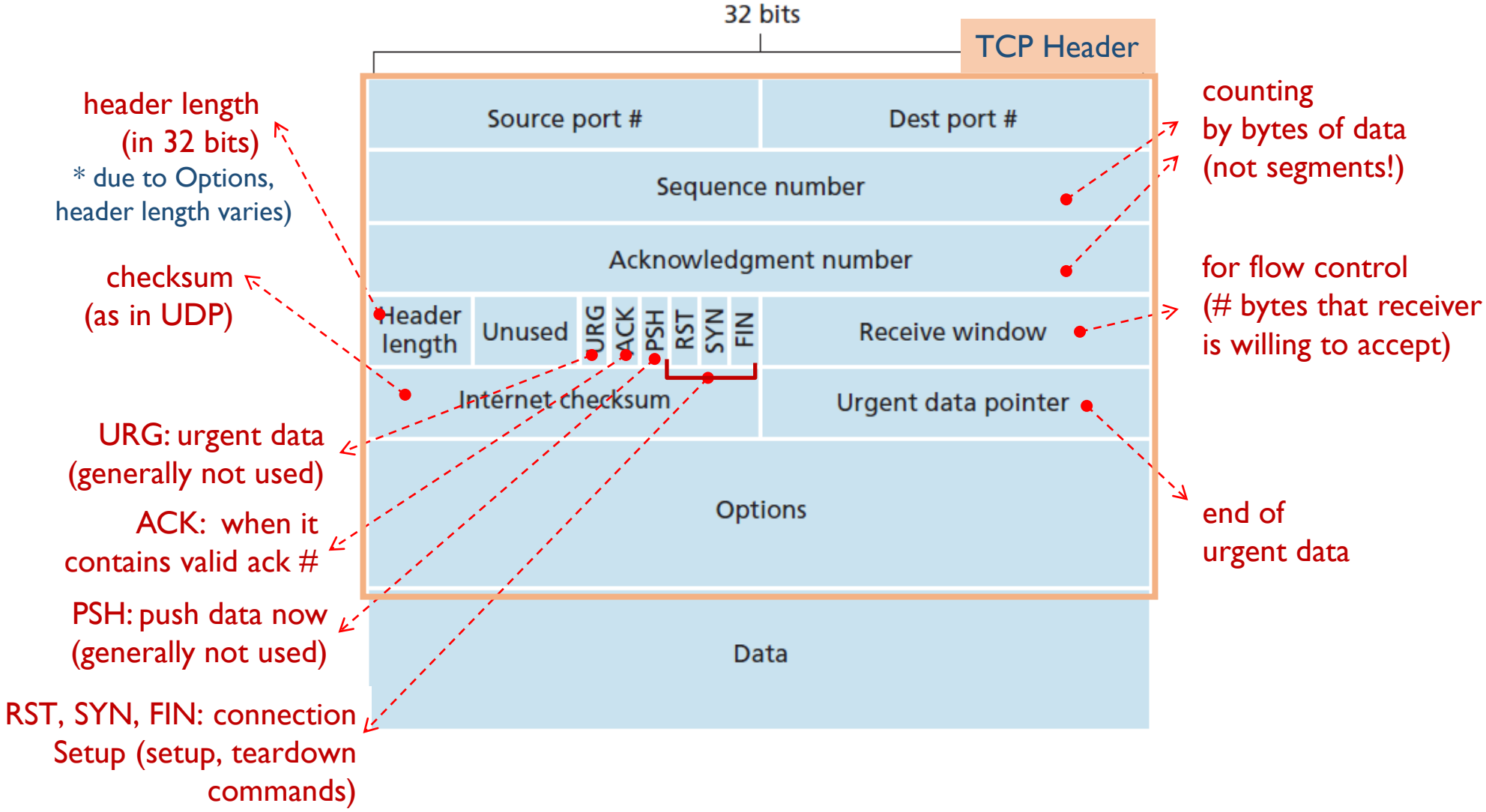
Connection between sender and receiver should be established before actual data delivery

To guarantee the reliability of data delivery, it should be checked whether data has been successfully delivered.

After the complete of data delivery, connection is released..



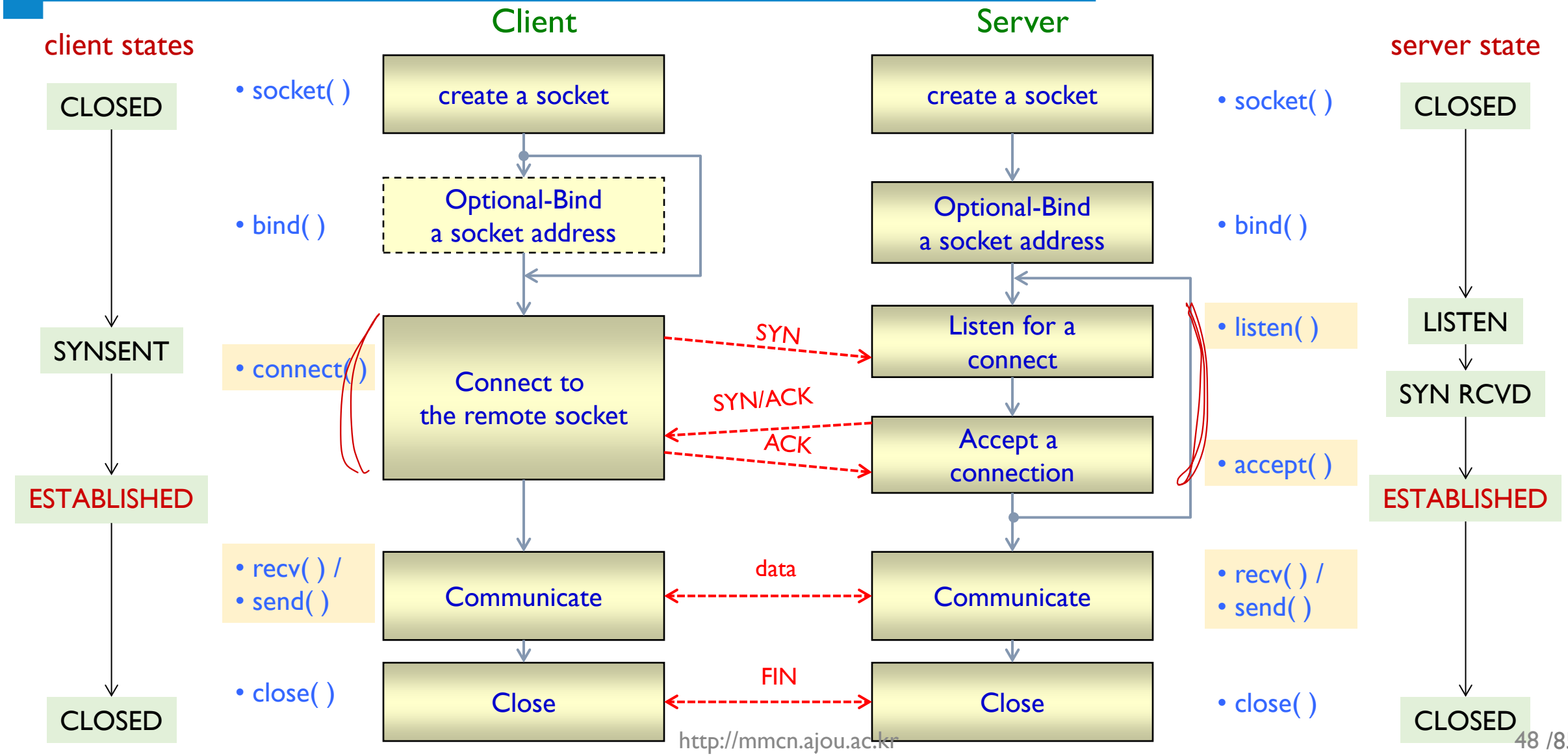
TCP – Header Structure



TCP Socket Programming

Socket Functions for TCP Applications

TCP Operation – Socket PGM View



connect( )

❖ connect( ) function – at a client

```
int connect ( int                socket,
              struct sockaddr    *foreignAddress,
              int                addressLength );
```

- arguments
  - `socket` : the socket file descriptor
  - `foreignAddress` : the server address that the program is connecting to
  - `addressLength` : the length of the server address (in bytes)
- return values
  - `0` : if successful
  - `-1` : if error
    - for the cause of the error, consult the value of `errno`

listen( ) (1/4)

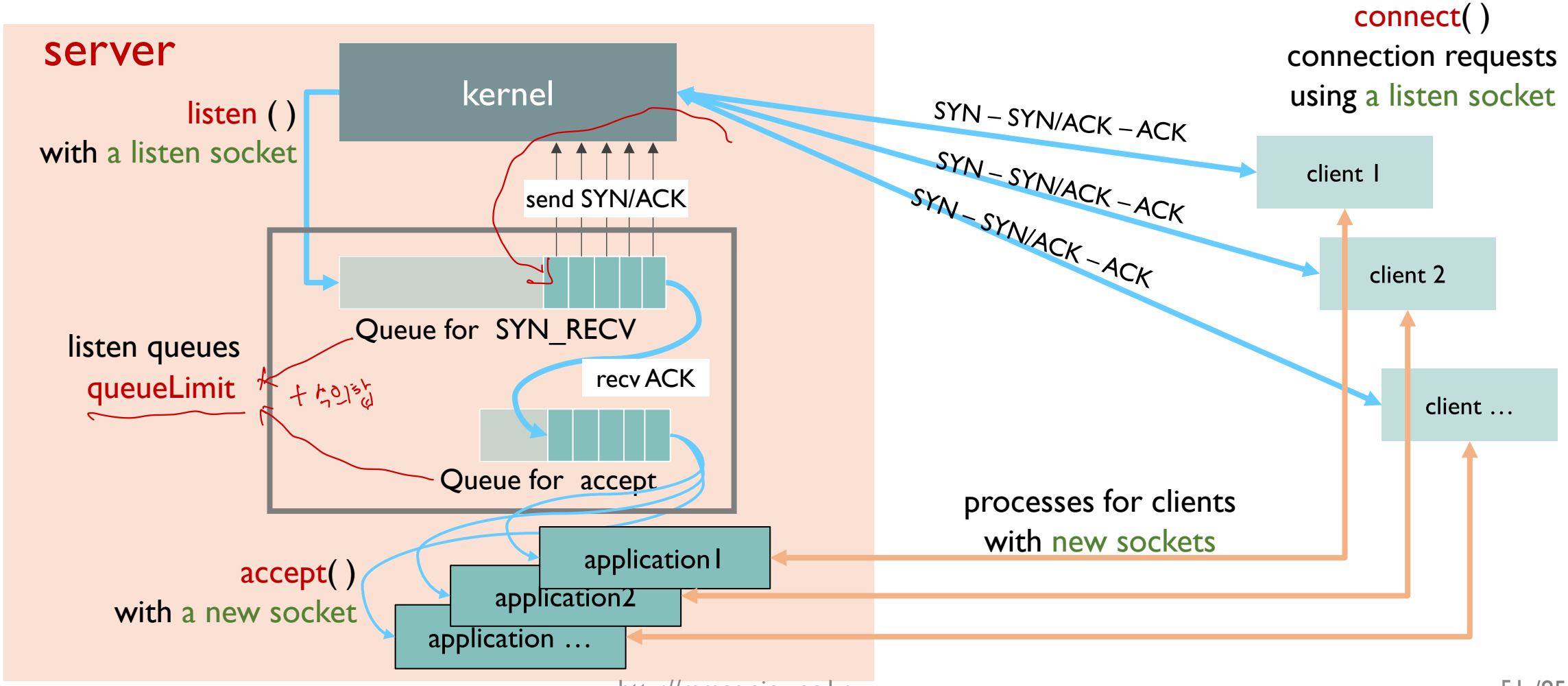
❖ listen( ) function – at a server

```
int listen ( int socket,      int queueLimit );
```

- arguments
  - socket : the socket descriptor
  - queueLimit : maximum number of connection requests waiting for accept( )
- return values
  - 0 : when successful
  - -1 : if error
    - for the cause of the error, consult the value of errno

listen( ) (2/4)

❖ listen queue and queueLimit

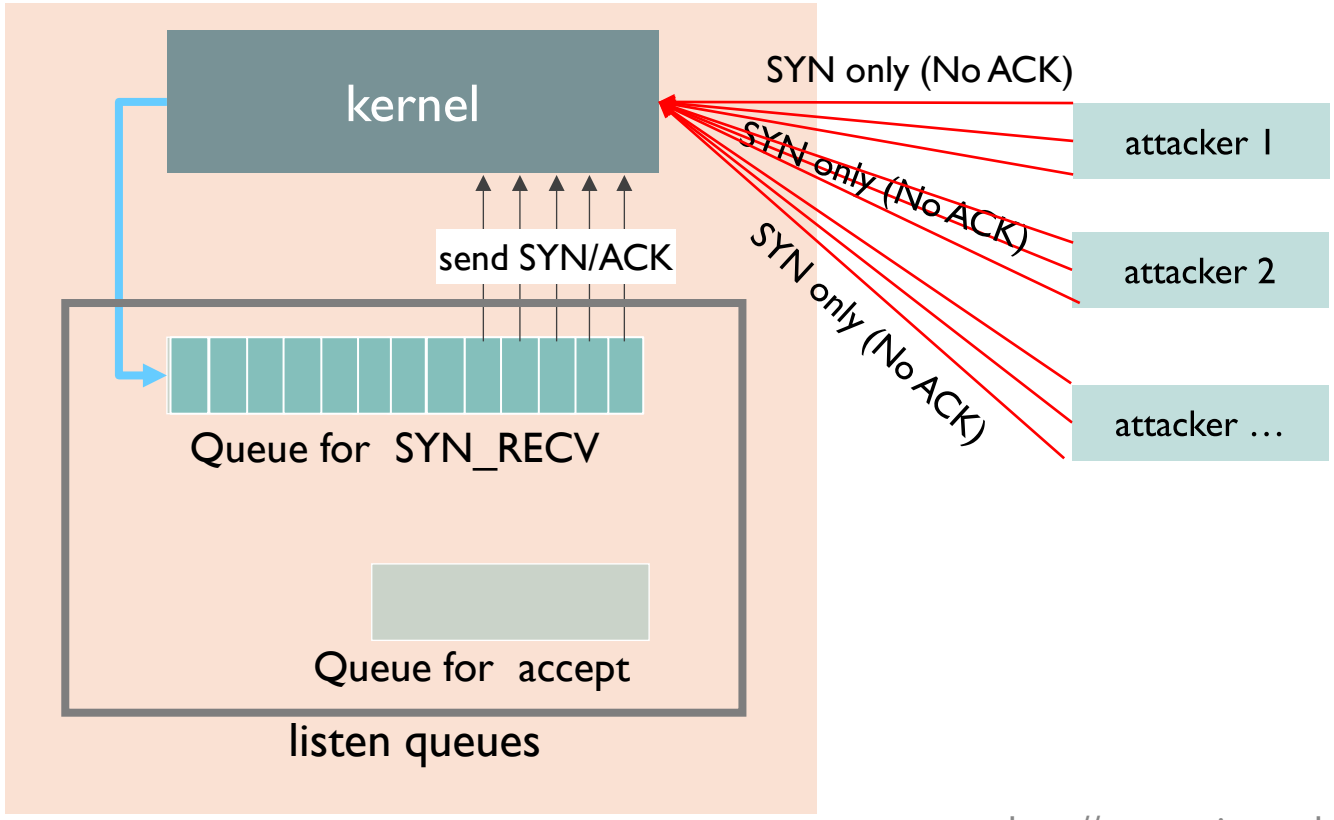


listen( ) (3/4)

- Specifying a value for *queueLimit*
  - it depends on the service types of the server
  - by heuristics
    - for small servers : 5 or more
    - for Web servers (about 45,000 connects per hour) : 16 or more
  - by default, 1024

listen( ) (4/4)

- SYN flooding attack
  - attacker send much number of SYN segment to a target in very short time duration, and no ACK for SYN-ACK




```
[root@net /root]# netstat -na|grep SYN
tcp        0      0 127.0.0.1:80            83.232.136.253:1911     SYN_RECV
tcp        0      0 127.0.0.1:80            199.106.229.9:1308      SYN_RECV
tcp        0      0 127.0.0.1:80            1.81.31.169:2051        SYN_RECV
tcp        0      0 127.0.0.1:80            47.95.74.60:1107        SYN_RECV
tcp        0      0 127.0.0.1:80            57.138.85.69:1697       SYN_RECV
tcp        0      0 127.0.0.1:80            54.125.11.117:2433      SYN_RECV
tcp        0      0 127.0.0.1:80            160.187.111.239:2064    SYN_RECV
tcp        0      0 127.0.0.1:80            167.129.136.239:2043    SYN_RECV
tcp        0      0 127.0.0.1:80            47.183.44.170:1256      SYN_RECV
tcp        0      0 127.0.0.1:80            124.215.160.217:1636    SYN_RECV
tcp        0      0 127.0.0.1:80            76.151.250.211:1673     SYN_RECV
tcp        0      0 127.0.0.1:80            95.168.224.46:1986      SYN_RECV
tcp        0      0 127.0.0.1:80            22.249.49.106:1362      SYN_RECV
tcp        0      0 127.0.0.1:80            21.255.176.15:1610      SYN_RECV
tcp        0      0 127.0.0.1:80            50.110.236.208:2272     SYN_RECV
tcp        0      0 127.0.0.1:80            114.218.24.238:1351     SYN_RECV
tcp        0      0 127.0.0.1:80            88.78.25.148:1215       SYN_RECV
tcp        0      0 127.0.0.1:80            104.254.58.198:2317     SYN_RECV
tcp        0      0 127.0.0.1:80            84.32.128.40:1065       SYN_RECV
tcp        0      0 127.0.0.1:80            244.164.245.106:1360    SYN_RECV
tcp        0      0 127.0.0.1:80            242.112.255.116:1032    SYN_RECV
tcp        0      0 127.0.0.1:80            130.134.146.84:2431     SYN_RECV
tcp        0      0 127.0.0.1:80            170.50.7.22:2020        SYN_RECV
tcp        0      0 127.0.0.1:80            157.180.196.126:1468    SYN_RECV
tcp        0      0 127.0.0.1:80            241.183.146.229:2024    SYN_RECV
tcp        0      0 127.0.0.1:80            105.145.93.35:1891      SYN_RECV
tcp        0      0 127.0.0.1:80            185.105.36.156:1881     SYN_RECV
```

accept( ) (1/2)

❖ accept( ) function – at a server

```
int accept ( int                socket,
             struct sockaddr    *clientAddress,
             int                *addressLength );
```

- arguments
  - `socket` : the input socket which is listening for connections
  - `clientAddress` : the pointer to a socket address structure
  - `addressLength` : the pointer to the maximum length of addr
    - note that it is a pointer to an integer data type, not integer value
- return values
  - `new socket value` : when successful
  - `-1` : if error
    - for the cause of the error, consult the value of errno

accept( ) (2/2)

- Note
  - accept( ) returns another new socket.
    - this is because any individual socket can only be connected to one client.

- two types of sockets used by the server program
  - listen socket
    - socket that is being used for listening
    - no reading or writing of data through theses sockets is permitted
  - client socket
    - socket that have been returned by accept( ) for individual client
    - it is connected to a client process, and
    - can be used to read and write data

send( ) and recv( ) (1/5)

TCP는 어디로 보내는지 알기에 sendto가 아닌 send

❖ send( ) function – send a message to the connected socket

```
int send ( int      socket,
           void     *msg,
           uint     msgLength,
           int      flags);
```

- arguments
  - `msg` : the pointer to the buffer containing message to send
  - `msgLength` : the length of `msg` to be sent (in bytes)
  - `flags` : the option for sending `msg` (to be discussed later)
- return values
  - `non-zero values` : the number of bytes written
    - it should match the supplied `msgLength` argument
    - however, it may be less than `msgLength` under valid circumstances
  - `-1` : if error
    - for the cause of the error, consult the value of `errno`

send( ) and recv( ) (2/5)

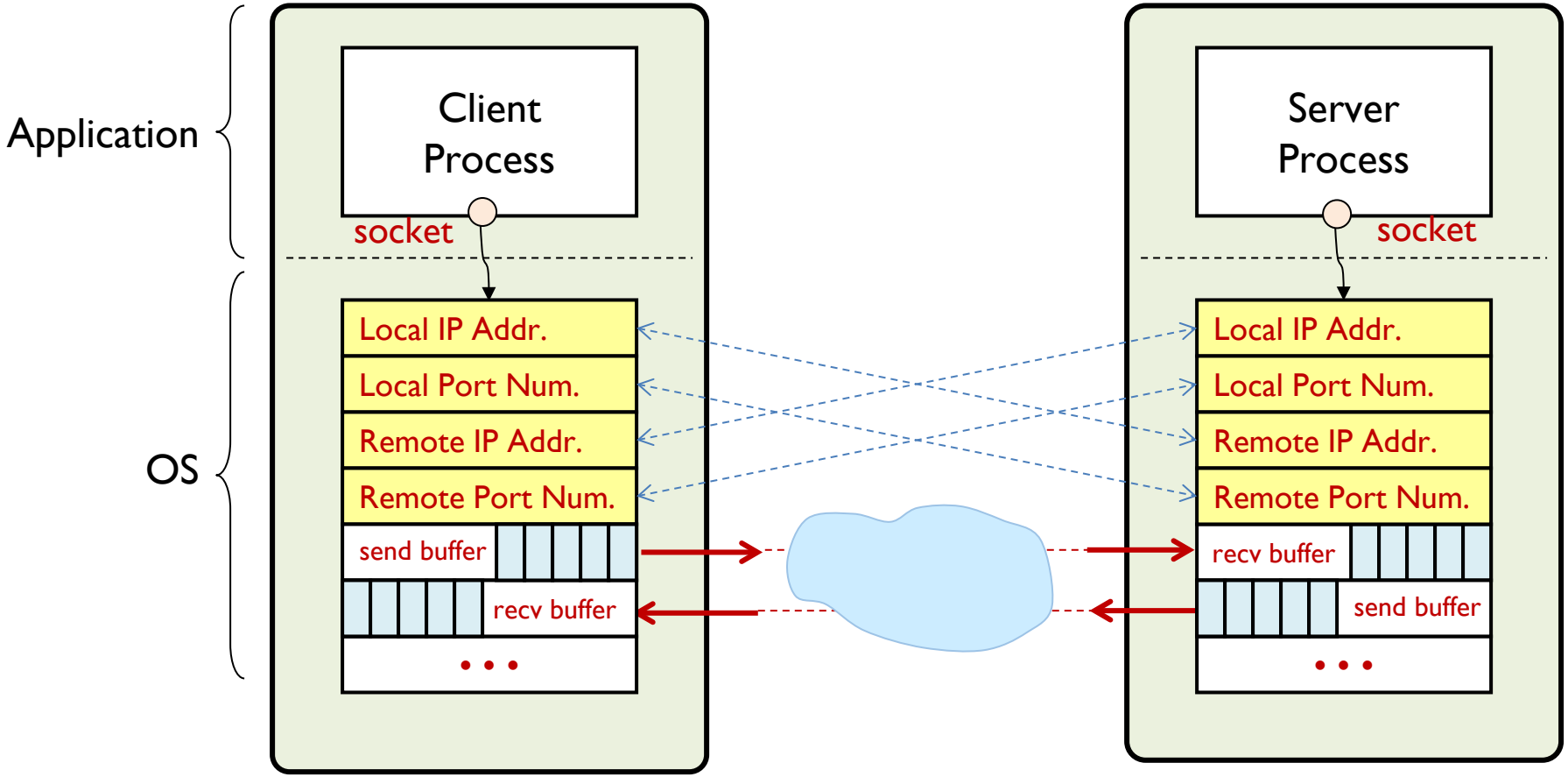
❖ recv( ) function – receive a message from the connected socket

```
int recv (int        socket,
          void       *rcvBuffer,
          uint       bufferLen,
          int        flags);
```

- arguments
  - `socket` : socket descriptor
  - `rcvBuffer` : the pointer to the receiving buffer memory
  - `bufferLen` : maximum size of `rcvBuffer` (in bytes)
  - `flags` : option for receiving data (to be discussed later)
- return values
  - `non-zero values` : the number of bytes read
  - `-1` : if error
    - for the cause of the error, consult the value of `errno`

send( ) and recv( ) (3/5)

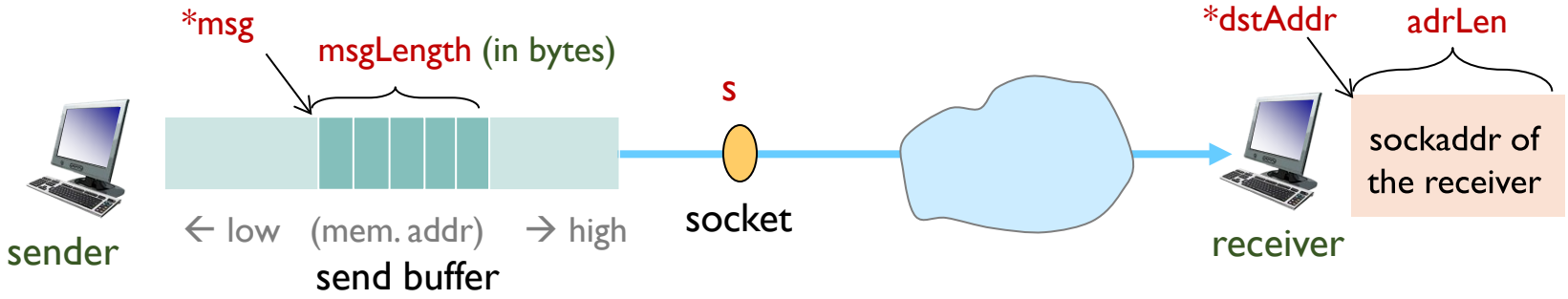
❖ TCP buffers



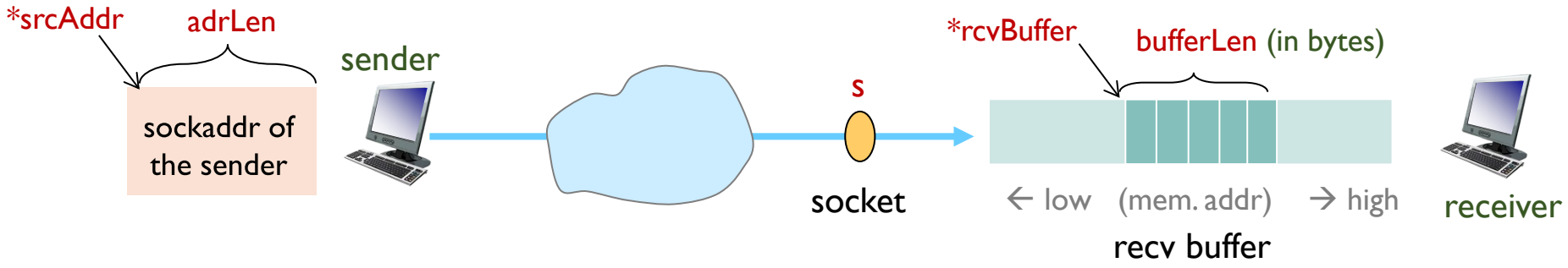
send( ) and recv( ) (4/5)

❖ send and recv buffers

- `int send (int s, void *msg, uint msgLength, int falgs);`



- `int recv (int s, void *rcvBuffer, uint bufferLen, int flags);`



send( ) and recv( ) (5/5)

- send() flags
  - MSG_CONFIRM
  - MSG_DONTROUTE - Don't use a gateway to send out the packet
  - MSG_DONTWAIT - Enables nonblocking operation
  - MSG_EOR
  - MSG_MORE
  - MSG_NOSIGNAL
  - MSG_OOB - sends out-of-band data on sockets
  - ...
- recv() flags
  - MSG_CMSG_CLOEXEC
  - MSG_DONTWAIT
  - MSG_ERRQUEUE
  - MSG_OOB
  - MSG_PEEK
  - MSG_TRUNC
  - MSG_WAITALL
  - ...

read( ) and write( ) (1/3)

❖ read( ) function - read a message from a file descriptor

```
ssize_t read (    int       fd,
                  void      *buf,
                  size_t    count );
```

- arguments
  - `fd` : the file descriptor (or socket descriptor)
  - `buf` : the pointer to the buffer memory
  - `count` : maximum size of `buf` (in bytes)
- return values
  - `0` : when end-of-file
  - `non-zero values` : the number of bytes read
  - `-1` : if error
    - for the cause of the error, consult the value of `errno`

read( ) and write( ) (2/3)

❖ write( ) function - write a message to a file descriptor

```
ssize_t write ( int          fd,
                const void   *buf,
                size_t       count );
```

- arguments
  - `fd` : the file descriptor (or socket descriptor)
  - `buf` : the pointer to the buffer memory
  - `count` : the length of data to be written in `buf` (in bytes)
- return values
  - `non-zero values` : the number of bytes written
    - it should match the supplied `count` argument
    - however, it may be less than `count` under valid circumstances
  - `-1` : if error
    - for the cause of the error, consult the value of `errno`

read( ) and write( ) (3/3)

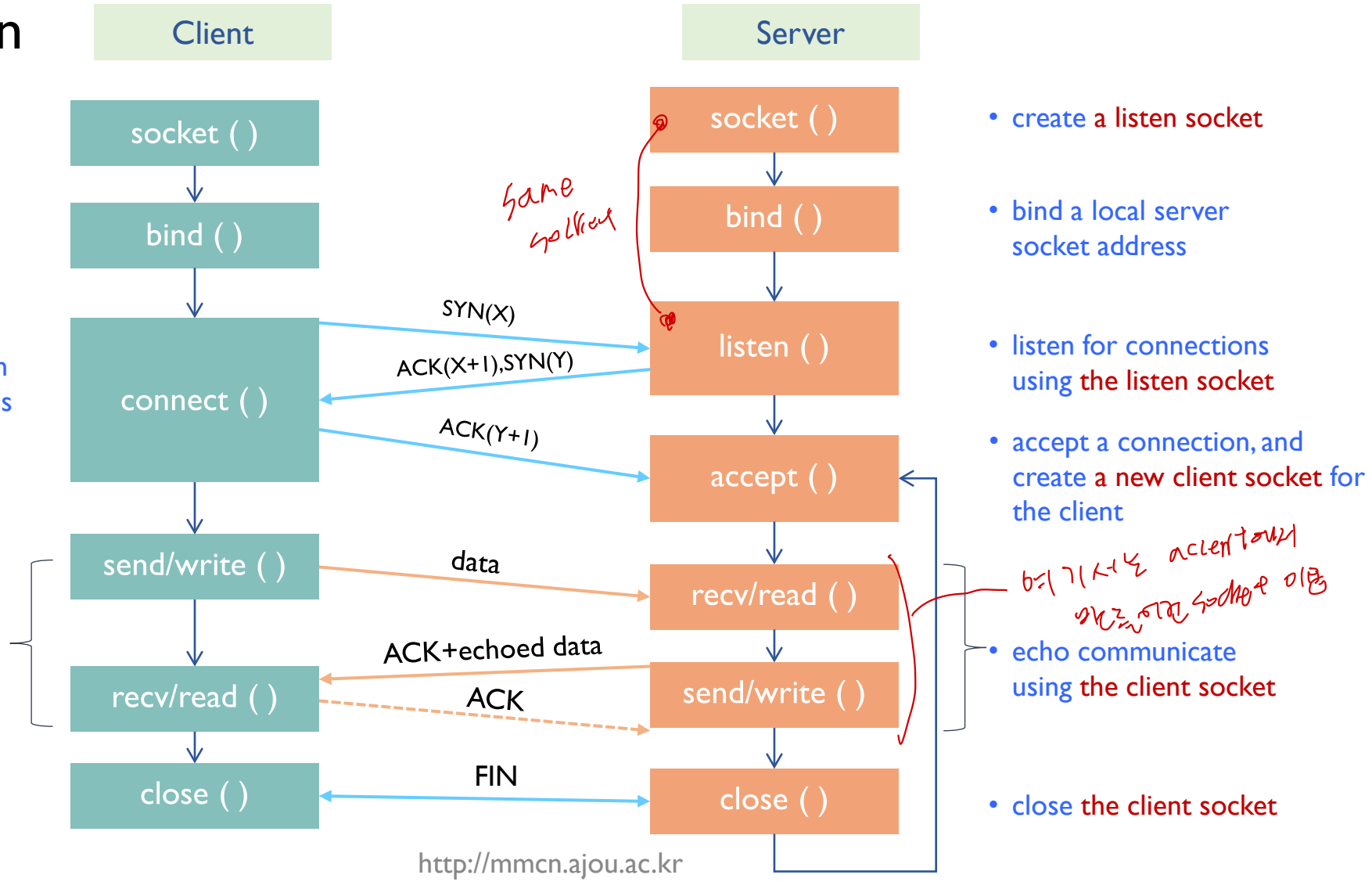
- What are differences between recv()/send() and read()/write() ?
  - read()/write()
    - the universal file descriptor functions working on all descriptors
  - recv()/send()
    - functions work only on socket descriptors
    - can specify certain options for the actual operations with flags

TCP Socket Programming

Example 2:TCP Echo Client-Server

TCP Echo Client-Server

❖ Operation



- create a socket
- bind a socket address (option)
- connection setup with remote server address and port
- echo communicate with the server
- close the socket

- create a listen socket
- bind a local server socket address
- listen for connections using the listen socket
- accept a connection, and create a new client socket for the client
- echo communicate using the client socket
- close the client socket

TCP Echo Client (1/3)

```
#include <winsock.h>
#define BUFSIZE 1500

int main(int argc, char* argv[])
{
    int                 retval, len;
    SOCKET              sock;
    SOCKADDR_IN         serveraddr;
    char                buf[BUFSIZE+1];

    WSADATA wsa;
    WSAStartup(MAKEWORD(2,2), &wsa) ;                       // winsock initialization

    sock = socket (AF_INET, SOCK_STREAM, 0);

    // server address
    ZeroMemory(&serveraddr, sizeof(serveraddr));
    serveraddr.sin_family = AF_INET;
    serveraddr.sin_port = htons(9000);                      // server port number (network order)
    serveraddr.sin_addr.s_addr = inet_addr("127.0.0.1");  // server IP address (network order)

    retval = connect (sock, (SOCKADDR *)&serveraddr, sizeof(serveraddr));
```

TCP Echo Client (2/3)

```
// communication with server
while (1) {
    // input data
    ZeroMemory(buf, sizeof(buf));
    printf("\n[보낼 데이터] ");
    fgets(buf, BUFSIZE+1, stdin) ;

    // remove the character '\n'
    len = strlen(buf);
    if(buf[len-1] == '\n')            buf[len-1] = '\0';

    // send data
    retval = send(sock, buf, strlen(buf), 0);
    printf("[TCP 클라이언트] %d바이트를 보냈습니다.\n", retval);

    // receive data
    retval = recv(sock, buf, BUFSIZE, 0);

    // display the received (echoed) data
    buf[retval] = '\0';
    printf("[TCP 클라이언트] %d바이트를 받았습니다.\n", retval);
    printf("[받은 데이터] %s\n", buf);
}
```

TCP Echo Client (3/3)

```
    // closesocket()
    closesocket (sock);

    // winsock end
    WSACleanup();
    return 0;

} // end of main
```

TCP Echo Server (1/3)

```
#include <winsock2.h>
#define BUFSIZE 1500

int main (int argc, char* argv[])
{
    SOCKET          listen_sock,  client_sock;// sockets for server & client
    SOCKADDR_IN     serveraddr, clientaddr;   // address for server & client
    int             addrlen, retval, msglen;
    char            buf[BUFSIZE+1];           // for receiving & sending data

    // winsock initialization
    WSADATA wsa;
    WSAStartup(MAKEWORD(2,2), &wsa) ;
    listen_sock = socket(AF_INET, SOCK_STREAM, 0);

    ZeroMemory(&serveraddr, sizeof(serveraddr));
    serveraddr.sin_family      = AF_INET;
    serveraddr.sin_port        = htons(9000);            // server listen port (network order)
    serveraddr.sin_addr.s_add  = htonl(INADDR_ANY);      // binding to any interface

    retval = bind(listen_sock, (SOCKADDR *)&serveraddr, sizeof(serveraddr));

    retval = listen(listen_sock, SOMAXCONN);
```

listen socket

TCP Echo Server (2/3)

client socket

```
// communication with clients
while(1) {
        addrlen = sizeof(clientaddr);
        client_sock = accept( listen_sock, (SOCKADDR *)&clientaddr, &addrlen);

        printf("\n[TCP Server] Client accepted : IP addr=%s, port=%d\n",
                                inet_ntoa(clientaddr.sin_addr), ntohs(clientaddr.sin_port));

        while(1) {
                // receive data from client
                msglen = recv(client_sock, buf, BUFSIZE, 0);

                // display received data
                buf[msglen] = '\0';
                printf("[TCP/%s:%d] %s\n",
                        inet_ntoa(clientaddr.sin_addr), ntohs(clientaddr.sin_port), buf);

                // echo received data to client
                retval = send(client_sock, buf, msglen, 0);
        }
        close(client_sock);
        printf("[TCP Server] Clinet termination: IP address=%s,  port=%d\n",
                        inet_ntoa (clientaddr.sin_addr), ntohs(clientaddr.sin_port));
}
```

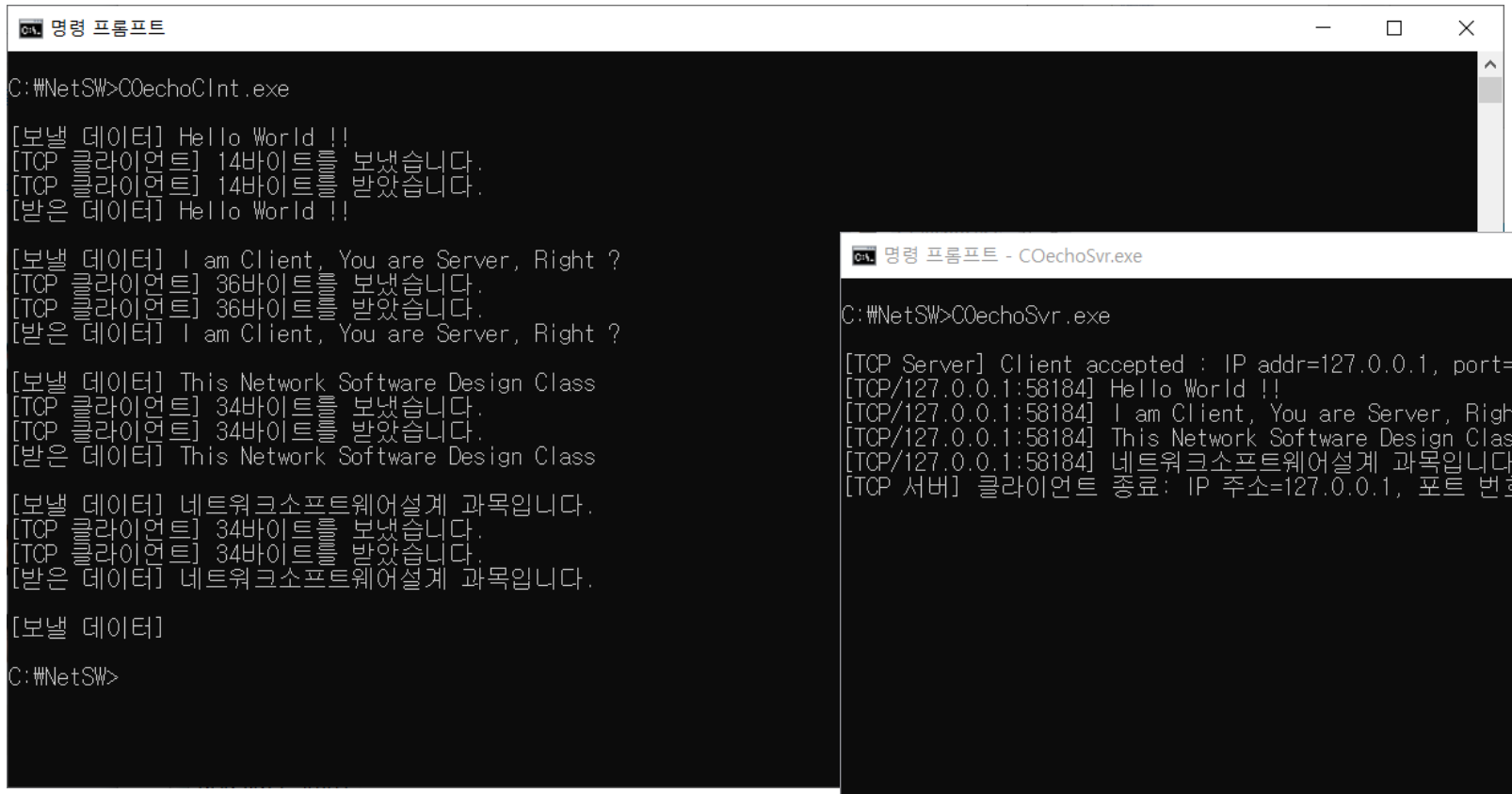
TCP Echo Server (3/3)

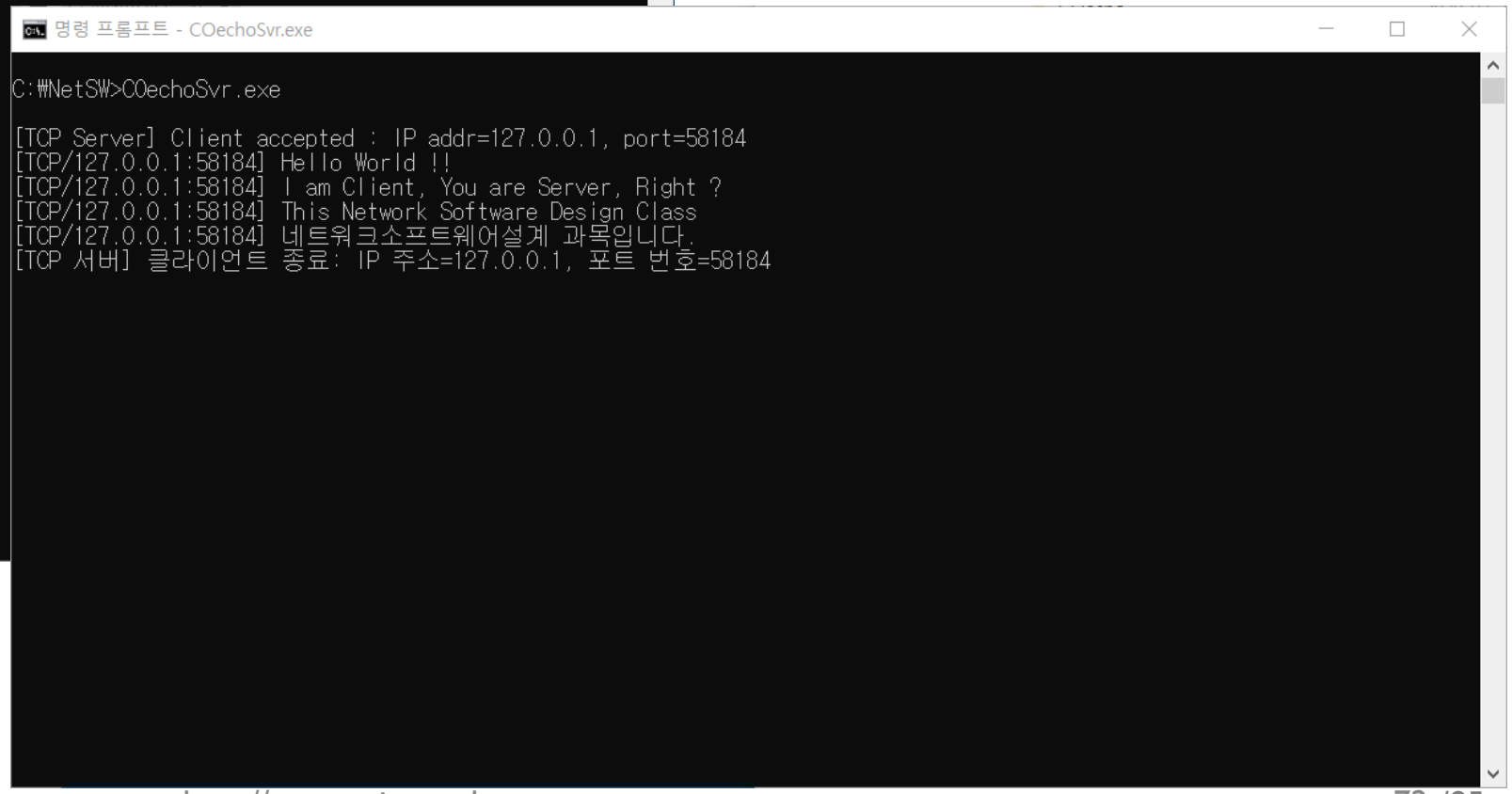
```
    // closesocket()
    closesocket(listen_sock);

    // winsock cleanup
    WSACleanup();
    return 0;

} // end of main
```

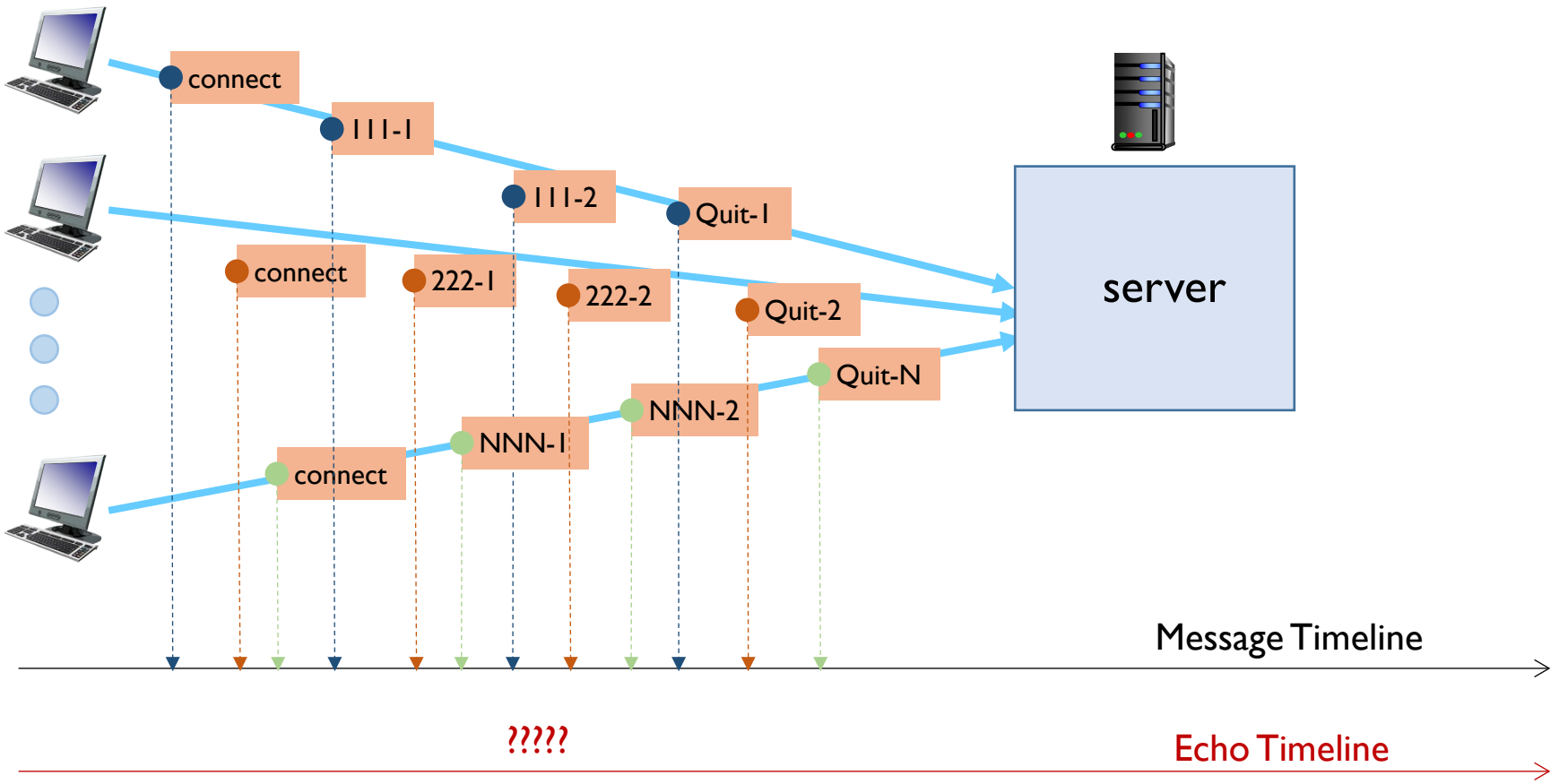
Run Example


명령 프롬프트
C:₩NetSW>COechoClnt.exe
[보낼 데이터] Hello World !!
[TCP 클라이언트] 14바이트를 보냈습니다.
[TCP 클라이언트] 14바이트를 받았습니다.
[받은 데이터] Hello World !!
[보낼 데이터] I am Client, You are Server, Right ?
[TCP 클라이언트] 36바이트를 보냈습니다.
[TCP 클라이언트] 36바이트를 받았습니다.
[받은 데이터] I am Client, You are Server, Right ?
[보낼 데이터] This Network Software Design Class
[TCP 클라이언트] 34바이트를 보냈습니다.
[TCP 클라이언트] 34바이트를 받았습니다.
[받은 데이터] This Network Software Design Class
[보낼 데이터] 네트워크소프트웨어설계 과목입니다.
[TCP 클라이언트] 34바이트를 보냈습니다.
[TCP 클라이언트] 34바이트를 받았습니다.
[받은 데이터] 네트워크소프트웨어설계 과목입니다.
[보낼 데이터]
C:₩NetSW>


명령 프롬프트 - COechoSvr.exe
C:₩NetSW>COechoSvr.exe
[TCP Server] Client accepted : IP addr=127.0.0.1, port=58184
[TCP/127.0.0.1:58184] Hello World !!
[TCP/127.0.0.1:58184] I am Client, You are Server, Right ?
[TCP/127.0.0.1:58184] This Network Software Design Class
[TCP/127.0.0.1:58184] 네트워크소프트웨어설계 과목입니다.
[TCP 서버] 클라이언트 종료: IP 주소=127.0.0.1, 포트 번호=58184

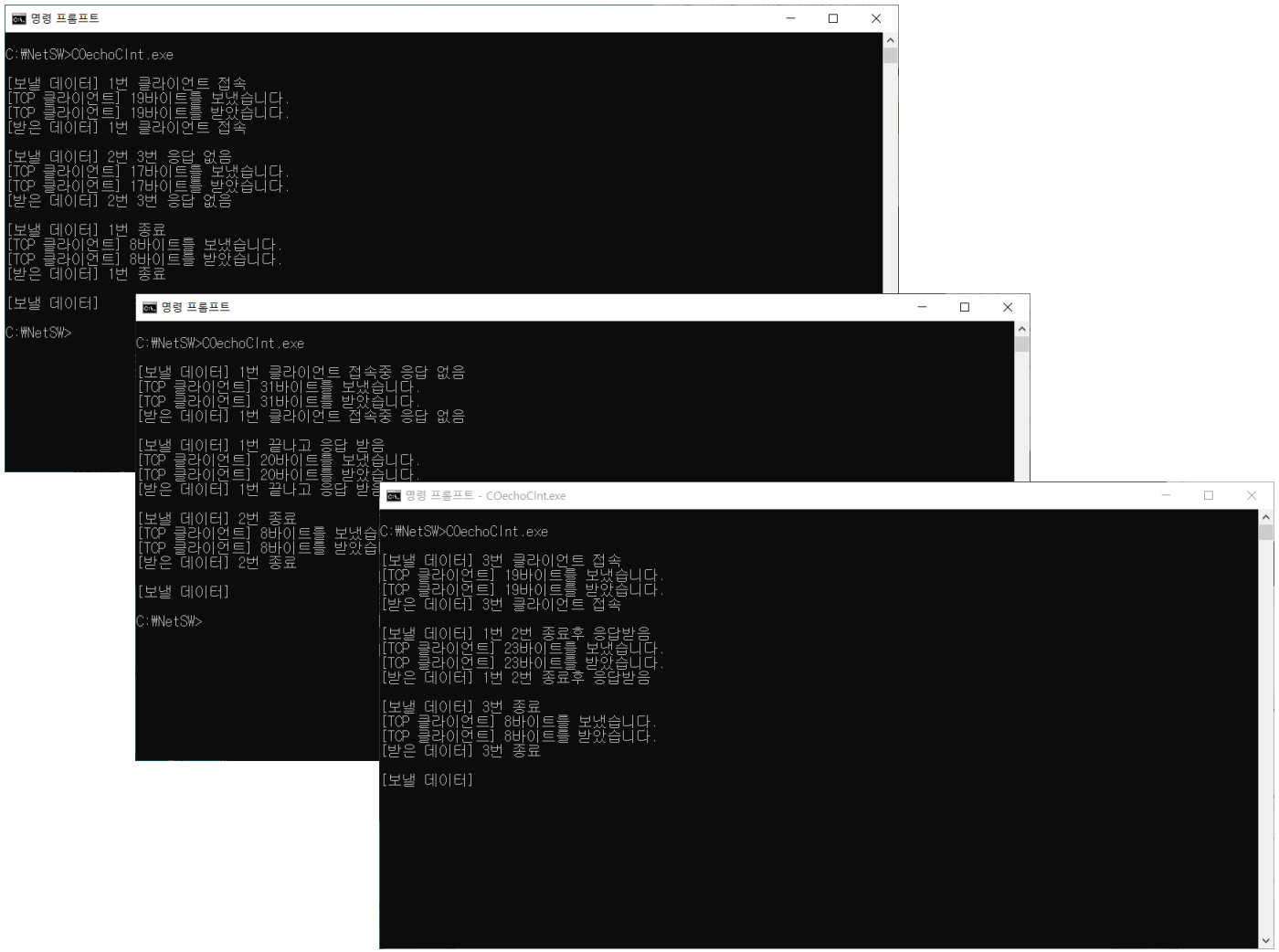
Discussion (1/3)

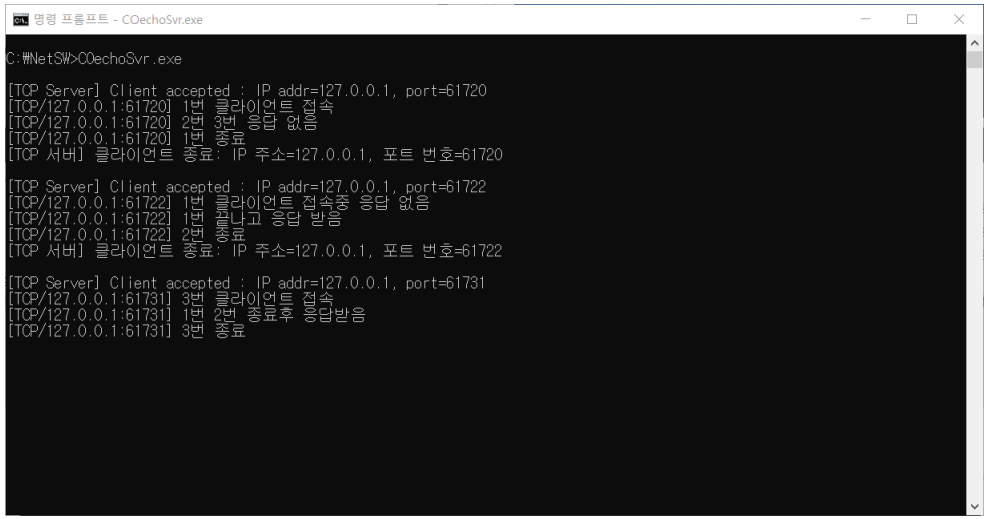
- For N >1 clients:

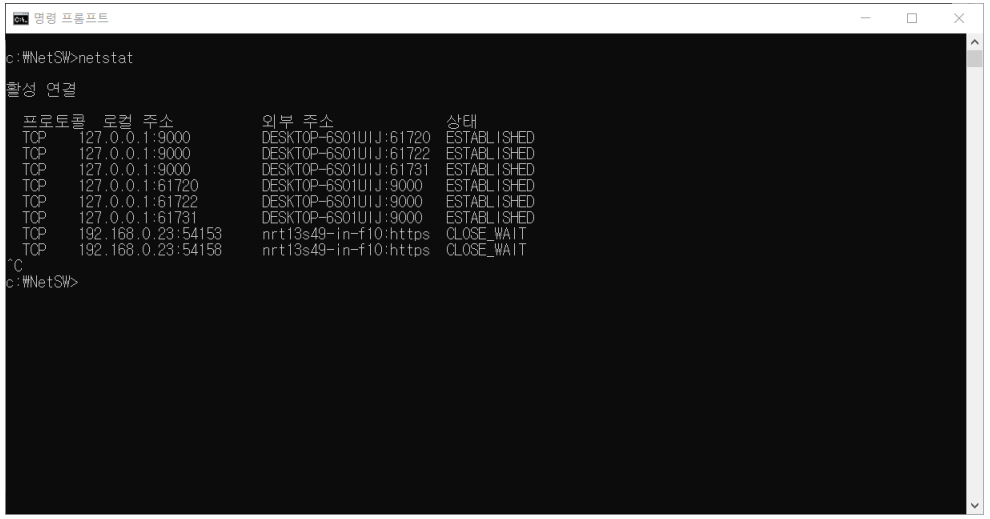


Discussion (2/3)

❖ Operation

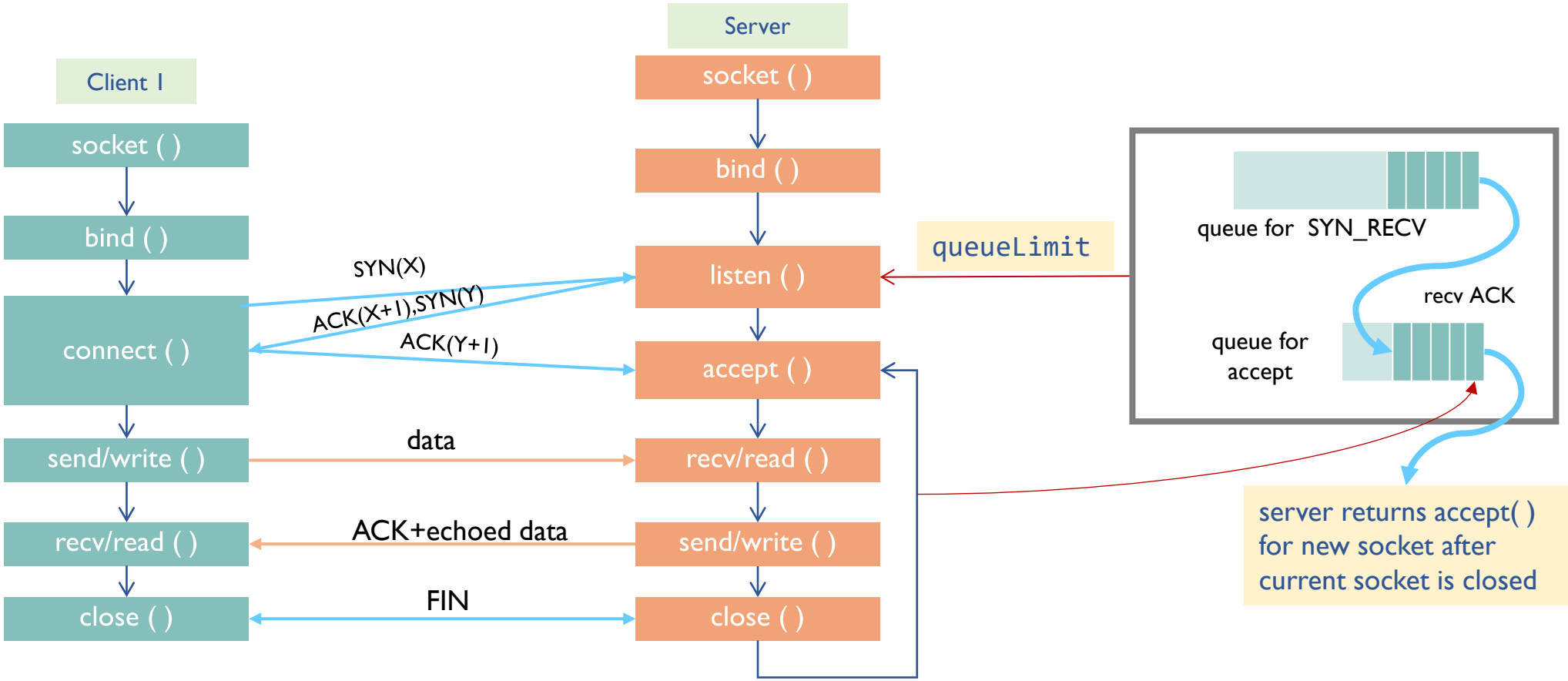






Discussion (3/3)

❖ Sequential Server



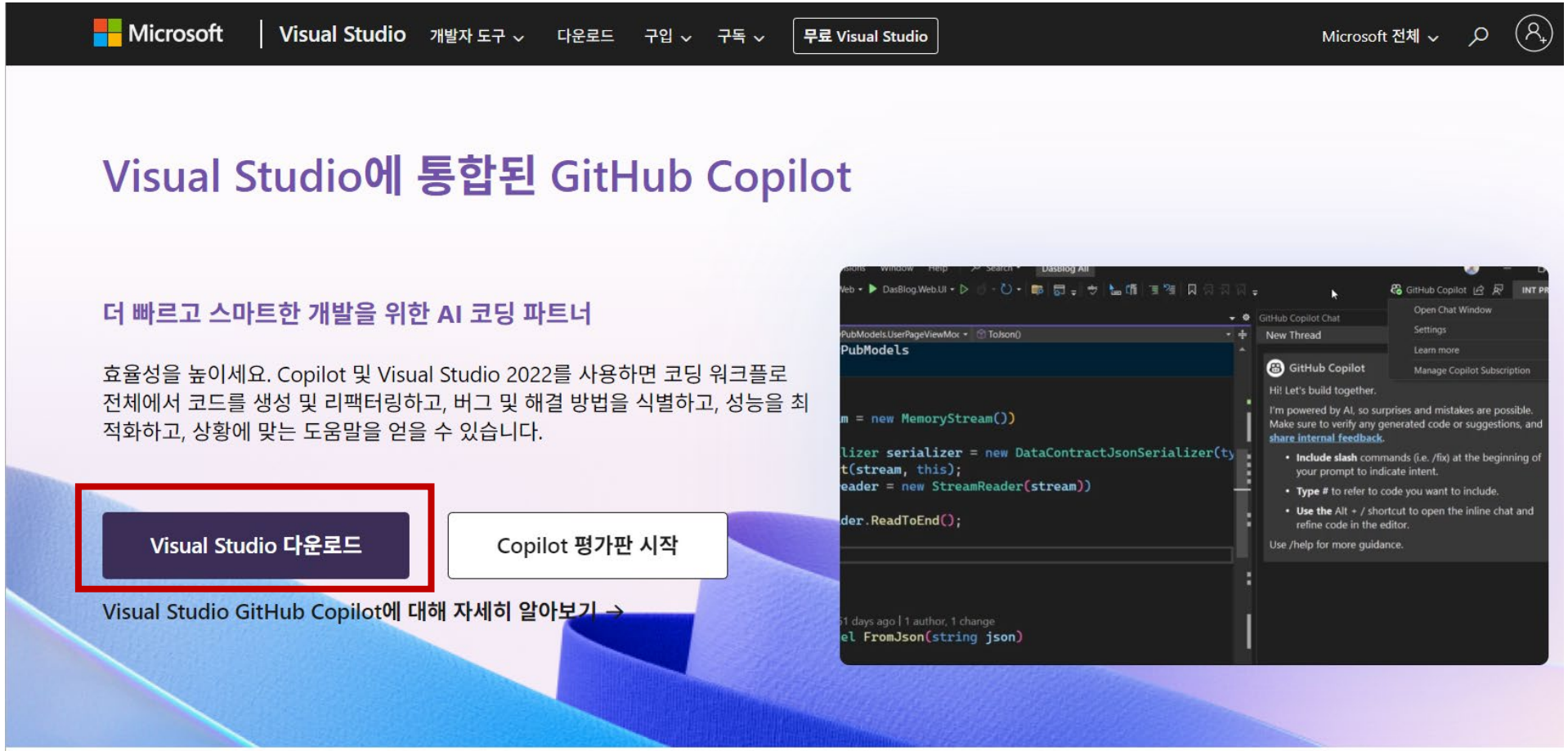
APPENDIX

Use of Visual Studio

MS Visual Studio

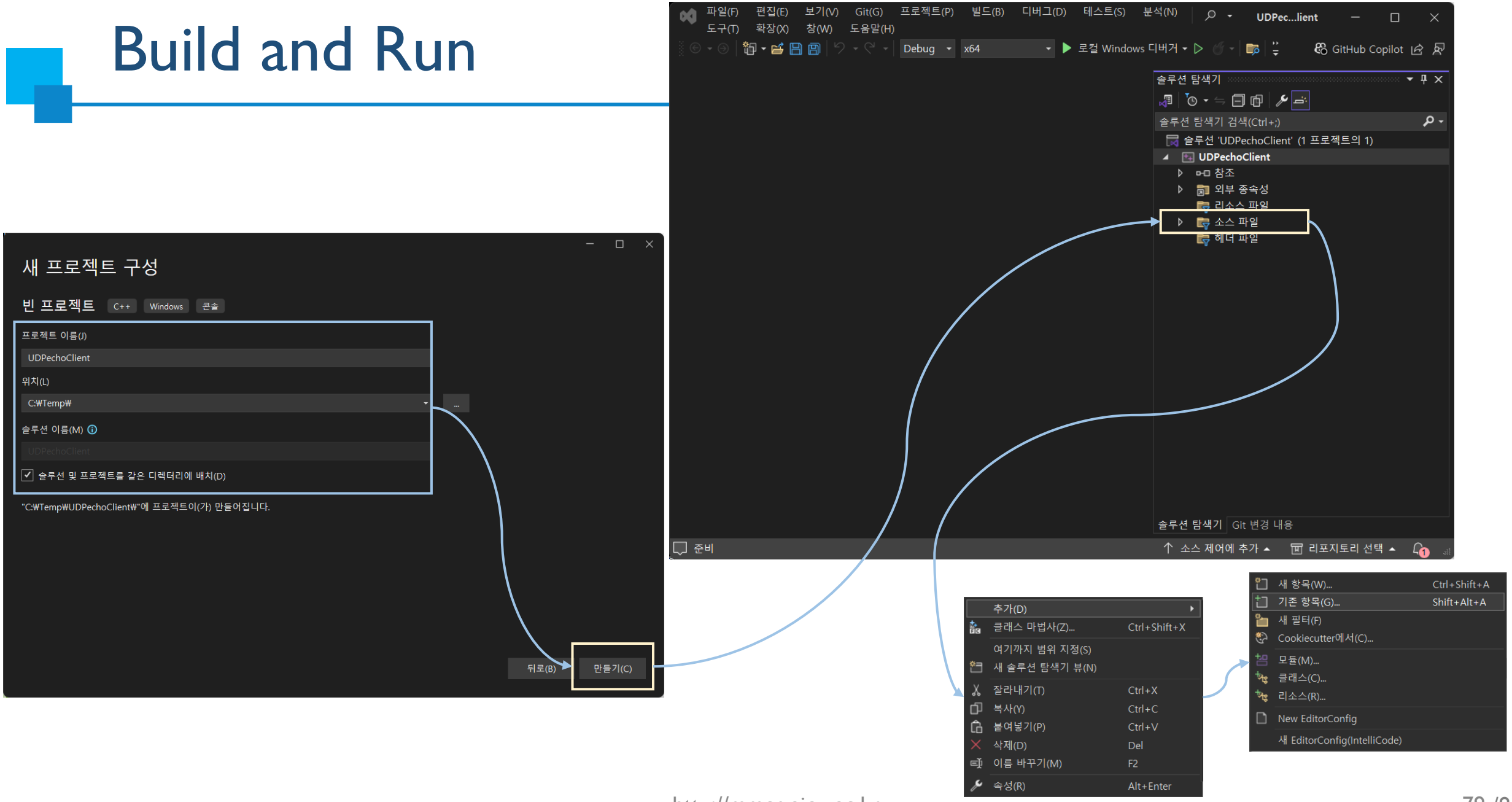
❖ MS Visual Studio 설치

- https://visualstudio.microsoft.com/ko/

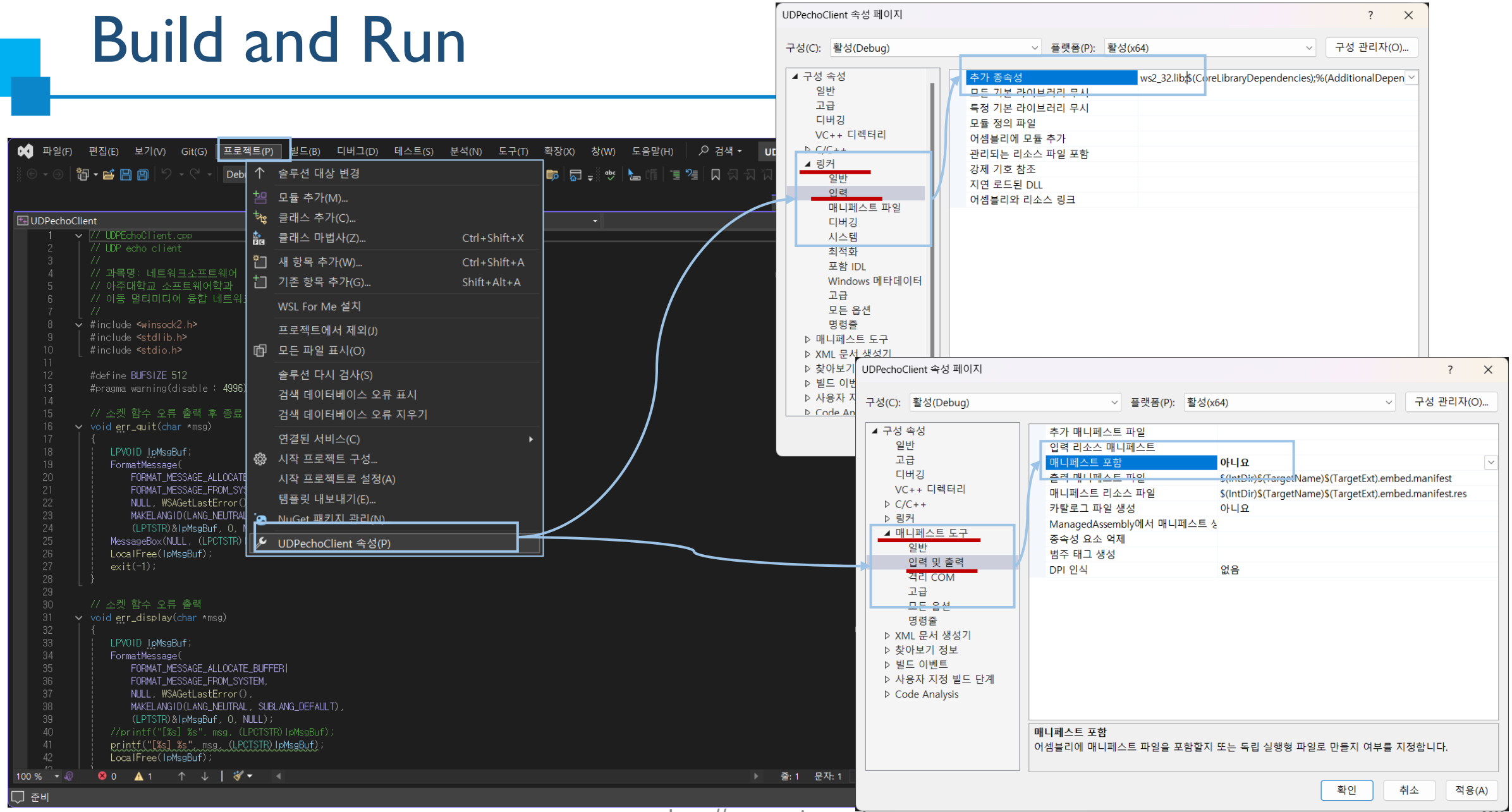


Build and Run

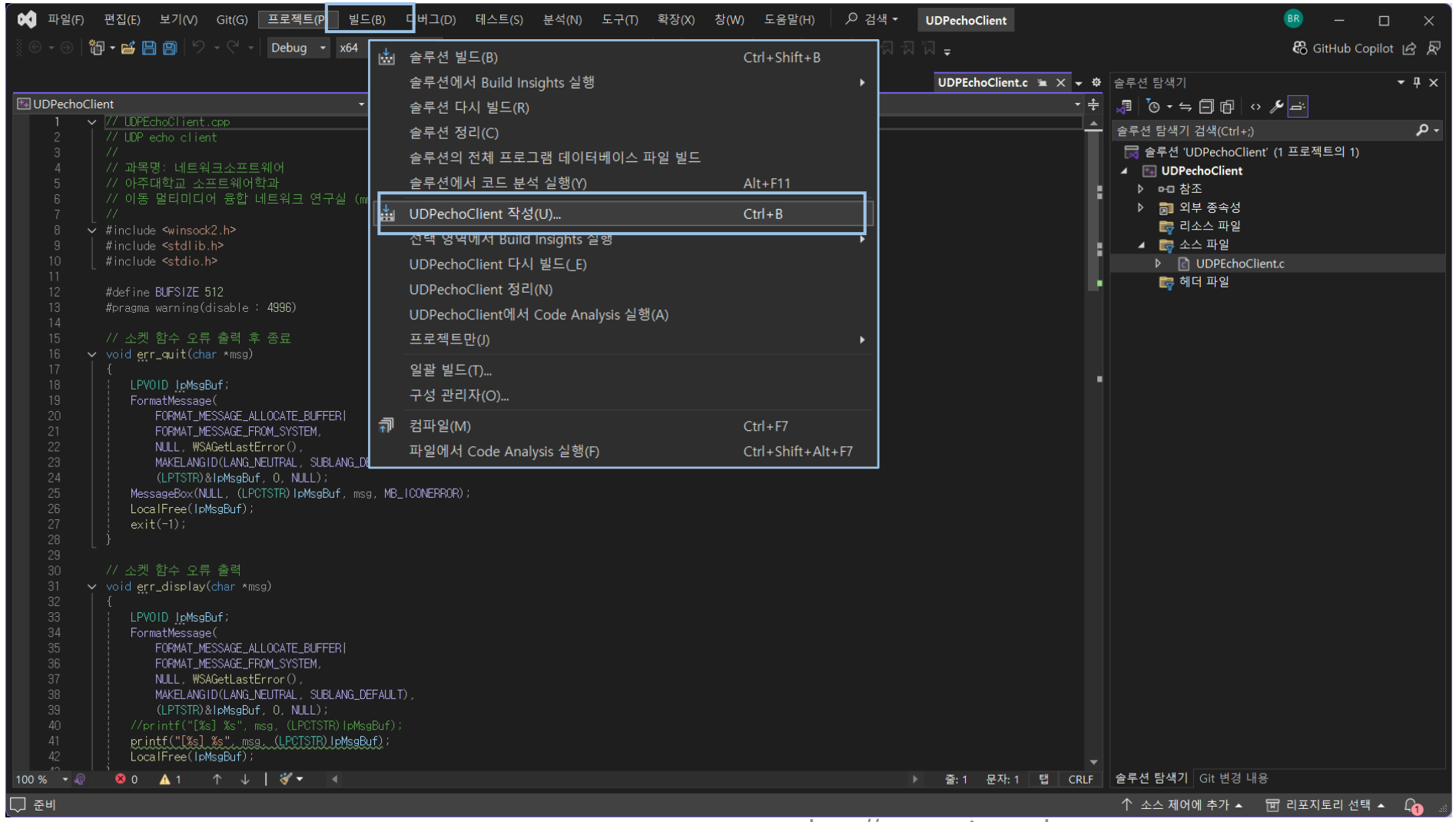
Build and Run



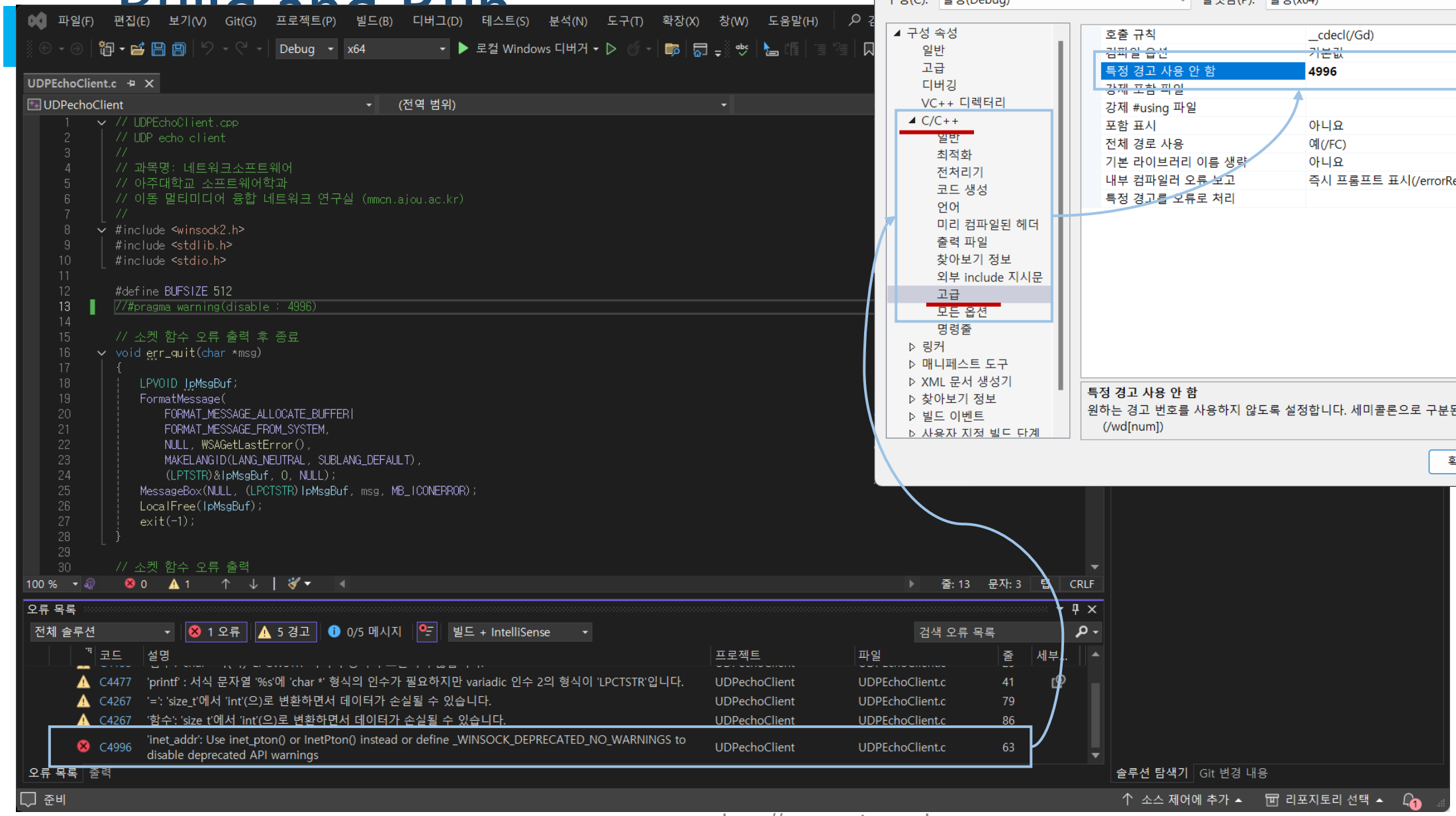
Build and Run

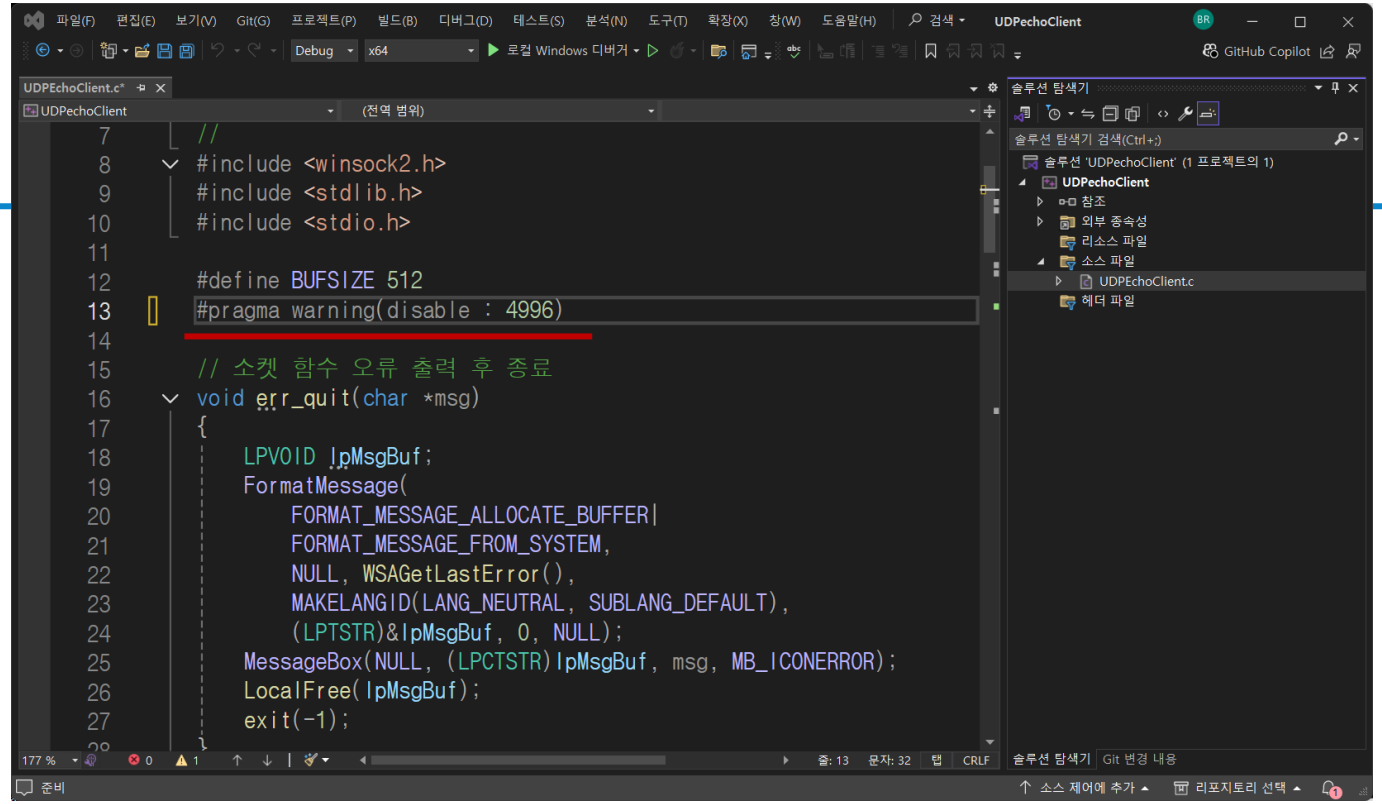


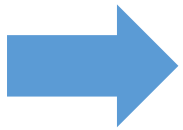
Build and Run

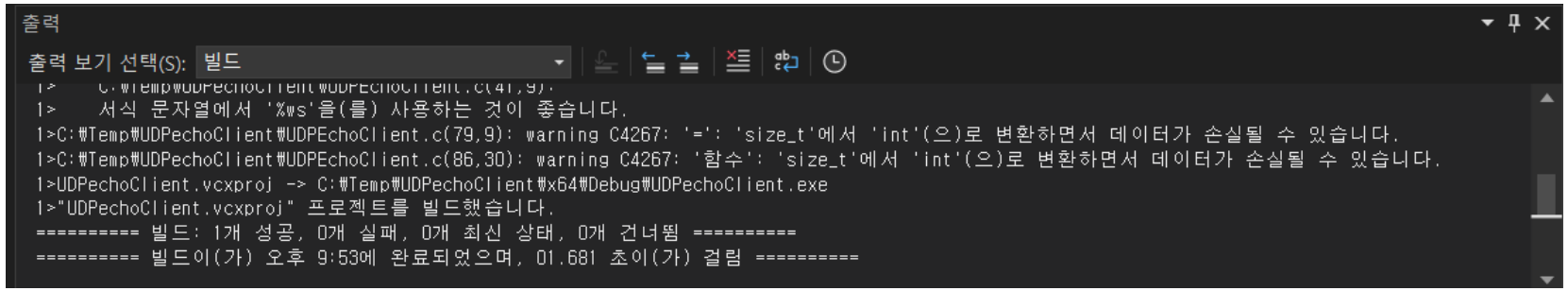


Build and Run




UDPechoClient.c*
UDPechoClient
(전역 범위)
//
#include <winsock2.h>
#include <stdlib.h>
#include <stdio.h>
#define BUFSIZE 512
#pragma warning(disable : 4996)
// 소켓 함수 오류 출력 후 종료
void err_quit(char *msg)
{
LPVOID lpMsgBuf;
FormatMessage(
FORMAT_MESSAGE_ALLOCATE_BUFFER|
FORMAT_MESSAGE_FROM_SYSTEM,
NULL, WSAGetLastError(),
MAKELANGID(LANG_NEUTRAL, SUBLANG_DEFAULT),
(LPTSTR)&lpMsgBuf, 0, NULL);
MessageBox(NULL, (LPCTSTR)lpMsgBuf, msg, MB_ICONERROR);
LocalFree(lpMsgBuf);
exit(-1);
솔루션 탐색기
솔루션 'UDPechoClient' (1 프로젝트의 1)
UDPechoClient
참조
외부 종속성
리소스 파일
소스 파일
UDPEchoClient.c
헤더 파일
준비




출력
출력 보기 선택(S): 빌드
1>   서식 문자열에서 '%ws'을(를) 사용하는 것이 좋습니다.
1>C:₩Temp₩UDPechoClient₩UDPEchoClient.c(79,9): warning C4267: '=': 'size_t'에서 'int'(으)로 변환하면서 데이터가 손실될 수 있습니다.
1>C:₩Temp₩UDPechoClient₩UDPEchoClient.c(86,30): warning C4267: '함수': 'size_t'에서 'int'(으)로 변환하면서 데이터가 손실될 수 있습니다.
1>UDPechoClient.vcxproj -> C:₩Temp₩UDPechoClient₩x64₩Debug₩UDPechoClient.exe
1>"UDPechoClient.vcxproj" 프로젝트를 빌드했습니다.
========== 빌드: 1개 성공, 0개 실패, 0개 최신 상태, 0개 건너뜀 ==========
========== 빌드이(가) 오후 9:53에 완료되었으며, 01.681 초이(가) 걸림 ==========

Run Example

명령 프롬프트

```
C:₩NetSW>COechoClnt.exe

[보낼 데이터] Hello World !!
[TCP 클라이언트] 14바이트를 보냈습니다.
[TCP 클라이언트] 14바이트를 받았습니다.
[받은 데이터] Hello World !!

[보낼 데이터] I am Client, You are Server, Right ?
[TCP 클라이언트] 36바이트를 보냈습니다.
[TCP 클라이언트] 36바이트를 받았습니다.
[받은 데이터] I am Client, You are Server, Right ?

[보낼 데이터] This Network Software Design Class
[TCP 클라이언트] 34바이트를 보냈습니다.
[TCP 클라이언트] 34바이트를 받았습니다.
[받은 데이터] This Network Software Design Class

[보낼 데이터] 네트워크소프트웨어설계 과목입니다.
[TCP 클라이언트] 34바이트를 보냈습니다.
[TCP 클라이언트] 34바이트를 받았습니다.
[받은 데이터] 네트워크소프트웨어설계 과목입니다.

[보낼 데이터]

C:₩NetSW>
```

명령 프롬프트 - COechoSvr.exe

```
C:₩NetSW>COechoSvr.exe

[TCP Server] Client accepted : IP addr=127.0.0.1, port=58184
[TCP/127.0.0.1:58184] Hello World !!
[TCP/127.0.0.1:58184] I am Client, You are Server, Right ?
[TCP/127.0.0.1:58184] This Network Software Design Class
[TCP/127.0.0.1:58184] 네트워크소프트웨어설계 과목입니다.
[TCP 서버] 클라이언트 종료: IP 주소=127.0.0.1, 포트 번호=58184
```

Questions ?

MR-IoT/AI Convergence

Future Internet & 5G/6G

Intelligent Networking with AI

MMcN

Mobile **Multimedia Convergence** Network Lab.

Homepage: http://mmcn.ajou.ac.kr

facebook: http://www.facebook.com/#!/groups/190702010947314

BK21 FOUR

AI융합네트워크학과